

TRANSFER CREDIT MATCH – WEBSITE CODE

Files included: 53

```
=====
=====
```

```
=====
=====
FILE: .gitignore
=====
=====
```

```
.venv/
__pycache__/
.pytest_cache/
*.pyc
*.pyo
*.pyd
```

```
frontend/node_modules/
frontend/dist/
```

```
backend/.env
frontend/.env
```

```
=====
=====
FILE: README.md
=====
=====
```

Transfer Credit Match – Data Science Source Package

This package provides backend/data-science workflow for transfer credit matching:

- parse transcript PDFs
- map courses to Knowledge Units (KUs)
- score source courses against destination institution courses (KU + title + credits + syllabus text)
- run transfer request workflows (submit, review, approve/deny)
- validate CSV data integrity
- export student transfer reports
- build a local SQLite database from seeded CSV files
- run tests for matcher and pipeline behavior

Project Layout

- `TranscriptPDF\_reader/pdf\_reader.py`: transcript parser
- `csv\_exports/`: seeded institutions/programs/courses/KU datasets

- `engine/data\_loader.py`: loads the CSV data bundle
- `engine/matcher.py`: KU + title + credit + syllabus scoring logic
- `engine/pipeline.py`: transcript-to-institution matching pipeline
- `engine/workflow.py`: submit + approve/deny workflow updates
- `engine/config.py`: institution-specific matching rule resolution
- `scripts/run\_transfer\_match.py`: main CLI for matching
- `scripts/build\_sqlite.py`: creates `transfer\_credit\_match.db` from CSVs
- `scripts/validate\_data.py`: validates key ID relationships across CSVs
- `scripts/workflow\_cli.py`: submit/review/report CLI
- `config/matching\_rules.json`: global + per-institution thresholds/weights
- `csv\_exports/course\_syllabus.csv`: optional syllabus snippets for similarity score
- `tests/`: starter automated tests

Setup

```
```bash
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```
```

Full Website Stack (Frontend + Backend)

You now have complete website code in:

- `backend/` -> FastAPI server
- `frontend/` -> React + Vite app

Fastest way (one terminal)

From the `Data\_Science` folder:

```
```bash
bash start_website.sh
```
```

Then open **http://localhost:5173** in your browser. The frontend proxies `/api` to the backend on port **8000** (see `frontend/vite.config.js`). You do not need to set `VITE\_API\_BASE\_URL` for local dev if `.env` is empty.

1) Start Backend API (manual)

```
```bash
cd backend
python3 -m venv .venv
```

```
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python run_api.py
````
```

Backend runs on `http://localhost:8000`

Useful endpoints:

```
- `GET /health`
- `GET /api/institutions`
- `GET /api/courses?institution_id=4`
- `POST /api/match/transcript` (multipart file upload)
- `POST /api/ai/explain` (API key-ready stub)
- `GET /api/ml/status`
- `POST /api/ml/train`
- `POST /api/ml/predict`
- `POST /api/ml/train-nlp`
- `POST /api/ml/predict-nlp`
- `POST /api/ml/analyze-kus`
```

2) Start Frontend Website

```
```bash
cd frontend
cp .env.example .env
npm install
npm run dev
```
```

Frontend runs on `http://localhost:5173`

API Key (Add Later)

When you are ready, add your key in `backend/.env`:

```
```env
OPENAI_API_KEY=your_key_here
```
```

The `/api/ai/explain` route is already wired to detect the key, so you can plug in real provider logic later without changing frontend wiring.

Your Custom ML Engine (Integrated)

Your provided script is now integrated at:

```
- `ml/transfer_matcher_full.py`
```

You can run it directly:

```
```bash
python ml/transfer_matcher_full.py train
python ml/transfer_matcher_full.py status
python ml/transfer_matcher_full.py predict '{"source":
[3,0,0,0],"target":[3,0,0,0]}'
python ml/transfer_matcher_full.py train_nlp --csv-dir csv_exports
python ml/transfer_matcher_full.py predict_nlp --source 1 --target 4
--csv-dir csv_exports
python ml/transfer_matcher_full.py analyze_kus --source 1 --target 4
--csv-dir csv_exports
```
```

And through the backend API:

```
```bash
curl -X POST http://localhost:8000/api/ml/predict \
 -H "Content-Type: application/json" \
 -d '{"source":[3,0,0,0],"target":[3,0,0,0]}'
```
```

Build Local SQLite Database

```
```bash
python scripts/build_sqlite.py --csv-root csv_exports --db-path
transfer_credit_match.db
```
```

Validate Dataset Integrity

```
```bash
python scripts/validate_data.py --csv-root csv_exports
```
```

Run Matching from a Transcript PDF

```
```bash
python scripts/run_transfer_match.py \
 --pdf "/absolute/path/to/transcript.pdf" \
 --csv-root csv_exports \
 --target-institution-id 4 \
 --top-k 3 \
 --threshold 0.55 \
 --rules config/matching_rules.json \
 --output match_output.csv
```
```

Output Columns

- `source\_course\_code`, `source\_course\_name`
- `target\_course\_code`, `target\_course\_name`
- `ku\_overlap\_count`, `ku\_overlap\_ratio`
- `title\_similarity`
- `syllabus\_similarity`
- `source\_credits`, `target\_credits`
- `score`
- `decision` (`Equivalent`, `Review`, `Not Equivalent`)

Scoring Rule (Configurable)

Default weighted score:

```
`score` = 0.60 * ku_overlap_ratio + 0.20 * title_similarity + 0.10 *
credit_ratio + 0.10 * syllabus_similarity`
```

- KU overlap is the strongest signal.
- Title, credit, and syllabus alignment refine confidence.
- You can tune thresholds and weights in `config/matching\_rules.json`.

Workflow Commands (No Website Needed)

Submit a transfer request and create a pending match:

```
```bash
python scripts/workflow_cli.py submit \
 --csv-root csv_exports \
 --student-id 1 \
 --institution-from 1 \
 --institution-to 4 \
 --course-from 1 \
 --course-to 4
```
```

Director review (approve or deny):

```
```bash
python scripts/workflow_cli.py review \
 --csv-root csv_exports \
 --match-id 1 \
 --changed-by 1 \
 --decision Approved
```
```

Export a student transfer report:

```
```bash
python scripts/workflow_cli.py report \
 --csv-root csv_exports \
```

```
--student-id 1 \
--output student_transfer_report.csv
```
```

```
## Testing
```

```
```bash
pytest -q
```
```

```
=====
=====
FILE: TranscriptPDF_reader/pdf_reader.py
=====
=====
```

```
import pdfplumber
import re
```

```
def extract_transcript_data(pdf_path):
    """
```

```
    Extract structured data from an unofficial academic transcript
    PDF.
```

```
    Works with Ellucian Banner transcripts from any institution (e.g.,
    Roosevelt University, Northeastern Illinois University, etc.)
    regardless
    of whether the PDF was exported directly or saved from the web
    self-service.
```

```
    Returns a dict with:
        - name: Student's full name
        - major: Declared major
        - minor: Declared minor (if any)
        - year: Academic year/classification (based on earned credit
        hours)
        - classes: List of dicts, each with {course, title, grade,
        credits, term}
    """
```

```
# — 1. Pull all raw text and word positions from every page
```

```
all_words = [] # list of {text, x0, x1, top, bottom, page}
with pdfplumber.open(pdf_path) as pdf:
    full_text = ""
    for page_num, page in enumerate(pdf.pages):
        page_text = page.extract_text()
        if page_text:
            full_text += page_text + "\n"
```

```

        # Also extract word-level data for position-based parsing
        for w in page.extract_words():
            all_words.append({
                'text': w['text'], 'x0': w['x0'], 'x1': w['x1'],
                'top': w['top'], 'bottom': w.get('bottom',
w['top'] + 10),
                'page': page_num
            })

```

— 2. Clean up web-page noise (Banner Self-Service saved as PDF)

```

# Remove URLs, page footers, timestamps
full_text = re.sub(r'https?:\/\/\S+', '', full_text)
full_text = re.sub(r'Page \d+ of \d+', '', full_text)

```

— 3. Extract INSTITUTION NAME

```

# Look for a line that contains a university/institute name.
# Skip section header lines like "College Major Minor".
institution = "Unknown"
# Patterns that indicate a real institution name (not a column
header)
inst_keywords = re.compile(
    r'\b(University|Institute|Polytechnic)\b', re.IGNORECASE
)
# Patterns for header labels we should skip
header_labels = re.compile(
    r'^(College\s+Major|Curriculum|Student|Transcript)',
re.IGNORECASE
)
for line in full_text.split('\n'):
    line = line.strip()
    if not line:
        continue
    if header_labels.match(line):
        continue
    if inst_keywords.search(line):
        # Clean up noise (timestamps, etc.)
        clean = re.sub(r'\d{1,2}/\d{1,2}/\d{2,4},?\s*\d{1,2}:
\d{2}\s*[AP]M', '', line).strip()
        if clean:
            institution = clean
            break
# Fallback: first non-boilerplate line
if institution == "Unknown":
    for line in full_text.split('\n'):
        line = line.strip()
        if (line and 'transcript' not in line.lower()
            and 'academic' not in line.lower()
            and not line.startswith('(')

```

```

        and not header_labels.match(line)):
            institution = line
            break

# — 4. Extract NAME


---


# Pattern 1: "Name Birth Date\n<FullName> <DD-MON>" (Banner
standard)
# Pattern 2: "<Name>,\n<First>." or "<Last>, <First>." header
style
name_match = re.search(
    r'Name\s+Birth\s+Date\n(.+)\s+\d{2}-[A-Z]{3}', full_text
)
if not name_match:
    # Web self-service format: "Name Birth Date" then name on next
line
    name_match = re.search(
        r'Name\s+Birth\s+Date\s*\n\s*(.+)\s+\d{2}-[A-Z]{3}',
full_text
    )
if not name_match:
    # Some transcripts: "<Last>, <First>\nStudent Academic
Transcript"
    name_match = re.search(
        r'^([A-Z][a-z]+,\s*[A-Z][a-z]+\.\s*)', full_text,
re.MULTILINE
    )
    name = name_match.group(1).strip() if name_match else "Unknown"

# — 5. Extract MAJOR and MINOR


---


# Try several patterns used by different Banner configurations:

# Pattern A: "Primary Degree\nMajor\n<value>" or
# "Primary Degree\nMajor Minor\n<value1> <value2>"
major = "Unknown"
minor = None

# Check for "Primary Degree" → "Major [Minor] [Major
Concentration]" block
primary_header = re.search(
    r'Primary\s+Degree\s*\n'
    r'(Major(?:\s+Minor)?(?:\s+Major\s+Concentration)?)\s*\n'
    r'(.+)',
    full_text
)
if primary_header:
    header_line = primary_header.group(1).strip()
    value_line = primary_header.group(2).strip()
    has_minor_field = 'Minor' in header_line

```

```

# Try splitting by 2+ spaces first (works for well-spaced
PDFs)
value_fields = re.split(r'\s{2,}', value_line)
if len(value_fields) >= 2 and has_minor_field:
    major = value_fields[0].strip()
    minor = value_fields[1].strip()
elif has_minor_field:
    # Spacing was lost – use word-level positions to split
columns.
    # Find the x-position of the "Minor" header word, then
split
    # the value row at that x-position.
    minor_x = None
    for w in all_words:
        if w['text'] == 'Minor':
            minor_x = w['x0']
            break
    if minor_x is not None:
        # Find all words on the value line (same vertical
position)
        # by finding words near the "Primary Degree" label's
y-position + offset
        major_words = []
        minor_words = []
        primary_y = None
        for w in all_words:
            if w['text'] == 'Primary':
                primary_y = w['top']
                break
        if primary_y is not None:
            # Value line is ~2 lines below the Primary Degree
header
            for w in all_words:
                # Check if this word is on the value line
(between header rows)
                if abs(w['top'] - (primary_y + 50)) < 20: #
approximate
                    pass # we'll use a different approach
                # More robust: find words that appear right after
"Primary Degree" → "Major Minor" line
                # Group all_words by approximate y-position (top)
from collections import defaultdict
                y_groups = defaultdict(list)
                for w in all_words:
                    # Round to nearest 2 pixels to group words on
the same line
                    y_key = round(w['top'] / 2) * 2
                    y_groups[y_key].append(w)
                # Find the line containing "Minor"

```

```

        minor_line_y = None
        for y_key, words in sorted(y_groups.items()):
            texts = [w['text'] for w in words]
            if 'Minor' in texts and 'Major' in texts:
                minor_line_y = y_key
                break
        if minor_line_y is not None:
            # The value line is the very next y-group
            sorted_ys = sorted(y_groups.keys())
            val_idx = sorted_ys.index(minor_line_y) + 1
            if val_idx < len(sorted_ys):
                val_line_words =
sorted(y_groups[sorted_ys[val_idx]], key=lambda w: w['x0'])
                major_words = [w['text'] for w in
val_line_words if w['x0'] < minor_x - 5]
                minor_words = [w['text'] for w in
val_line_words if w['x0'] >= minor_x - 5]
                if major_words:
                    major = ' '.join(major_words)
                if minor_words:
                    minor = ' '.join(minor_words)
            if major == "Unknown":
                # Last resort: take the whole line as major
                major = value_line
        else:
            major = value_fields[0].strip()

    if major == "Unknown":
        # Pattern B: "College Major [Minor]\n<college> <major>
[<minor>]"
        col_header = re.search(
            r'College\s+Major(?:\s+(Minor|Concentration))?\s*\n'
            r'(.+)',
            full_text
        )
        if col_header:
            has_minor_col = col_header.group(1) is not None
            value_line = col_header.group(2).strip()
            parts = re.split(r'\s{2,}', value_line)
            # First part is college name, second is major
            if len(parts) >= 2:
                major = parts[1].strip()
            if has_minor_col and len(parts) >= 3:
                minor = parts[2].strip()

    if major == "Unknown":
        # Pattern C: "Major\n<value>" standalone
        major_match = re.search(r'\bMajor\s*\n\s*(.+)', full_text)
        if major_match:
            major = major_match.group(1).strip()

```

— 6. Determine YEAR (classification)

```
# Find "Overall" totals line – works across institutions:
# "Overall 108.000 108.000 108.000 105.00 388.00 3.70"
earned_hours = 0.0
# Try multiple patterns for the "Overall" totals line
overall_match = re.search(
    r'Overall\s+([\d.]+)\s+([\d.]+)\s+([\d.]+)\s+([\d.]+)',
full_text
)
if overall_match:
    # 3rd number is typically "Earned Hours"
    earned_hours = float(overall_match.group(3))
else:
    # Fallback: look for cumulative totals in the last term block
    cum_matches = list(re.finditer(
        r'Cumulative\s+([\d.]+)\s+([\d.]+)\s+([\d.]+)', full_text
    ))
    if cum_matches:
        last_cum = cum_matches[-1]
        earned_hours = float(last_cum.group(3))

if earned_hours >= 90:
    year = "Senior"
elif earned_hours >= 60:
    year = "Junior"
elif earned_hours >= 30:
    year = "Sophomore"
elif earned_hours > 0:
    year = "Freshman"
else:
    year = "Unknown"
```

— 7. Extract COURSES

```
classes = []

# --- 7a. Transfer credits ---
# Find transfer credit sections (case-insensitive)
transfer_section = re.search(
    r'TRANSFER\s+CREDIT\s+ACCEPTED\s+BY\s+INSTITUTION\s*\n'
    r'(.+?)'
    r'(?=INSTITUTION\s+CREDIT|Transcript\s+Totals|
TRANSCRIPT\s+TOTALS|$)',
full_text, re.DOTALL | re.IGNORECASE
)
if transfer_section:
    transfer_text = transfer_section.group(1)
```

```

# Transfer courses: SUBJECT COURSE TITLE GRADE CREDITS
QUALITY_POINTS
    transfer_pattern = re.compile(
        r'^([A-Z]{2,5})\s+'           # Subject (2-5
uppercase letters)
        r'(\d{1,3}[A-Z]{0,3})\s+'   # Course number
(e.g., 210, 3XX, 1XX)
        r'(.+?)\s+'                 # Title
        r'([A-F][+-]?|P|TCR)\s+'    # Grade
        r'(\d+\.\d{3})\s+'          # Credit hours
        r'(\d+\.\d{2})',            # Quality points
        re.MULTILINE
    )
    for match in transfer_pattern.finditer(transfer_text):
        title = match.group(3).strip()
        # Skip header lines that got caught
        if title in ('Title', 'Subject') or 'Attempt Hours' in
title:
            continue
        classes.append({
            'course': f"{match.group(1)} {match.group(2)}",
            'title': title,
            'grade': match.group(4).strip(),
            'credits': float(match.group(5)),
            'term': "Transfer",
        })

# --- 7b. Institution (completed) courses ---
# Split by "Term:" or "Term :" headers
term_blocks = re.split(r'(?=Term\s*:\s+)', full_text)

# Locate where "COURSE(S) IN PROGRESS" section starts
in_progress_idx = full_text.upper().find("COURSE(S) IN PROGRESS")

for block in term_blocks:
    term_match = re.search(r'Term\s*:\s+(.+)', block)
    if not term_match:
        continue
    term_name = term_match.group(1).strip()

    # Determine if this block is inside the in-progress section
    block_start = full_text.find(block)
    is_in_progress = (in_progress_idx != -1 and block_start >=
in_progress_idx)

    if is_in_progress:
        # In-progress courses - various formats:
        # Format 1 (Roosevelt): "SUBJECT COURSE CAMPUS LEVEL TITLE
CREDITS"
        # Format 2 (NEIU):      "SUBJECT COURSE LEVEL TITLE

```

```

CREDITS"
    ip_pattern = re.compile(
        r'^([A-Z]{1,5})\s+'          # Subject
        r'(\d{1,3}[A-Z]?)\s+'      # Course number
        r'(?:(?:CHICAGO\s+CAMPUS|ONLINE(?:\s+CAMPUS)?|false)
\s+)?)' # Optional campus
        r'(?:U(?:G)?|G)\s+'        # Level (U, UG,
G)
        r'(.+)\s+'                  # Title
        r'(\d+\.\d{3})\s*$',       # Credits
        re.MULTILINE
    )
    for match in ip_pattern.finditer(block):
        title = match.group(3).strip()
        if title in ('Title',) or 'Attempt' in title:
            continue
        classes.append({
            'course': f"{match.group(1)} {match.group(2)}",
            'title': title,
            'grade': "In Progress",
            'credits': float(match.group(4)),
            'term': term_name,
        })

    # Some transcripts have in-progress courses with broken
line wrapping
    # where credits appear on same line as subject: "ECON 217
UG Principles.000"
    # We handle those by a more relaxed fallback
    if not any(c['term'] == term_name for c in classes):
        ip_fallback = re.compile(
            r'^([A-Z]{1,5})\s+'
            r'(\d{1,3}[A-Z]?)\s+'
            r'(?:U(?:G)?|G)\s+'
            r'(.+)\s+(\d+\.\d{3})',
            re.MULTILINE
        )
        for match in ip_fallback.finditer(block):
            title = match.group(3).strip()
            classes.append({
                'course': f"{match.group(1)}
{match.group(2)}",
                'title': title,
                'grade': "In Progress",
                'credits': float(match.group(4)),
                'term': term_name,
            })
    else:
        # Completed courses with grades
        # This pattern covers multiple campus/level formats:

```

```

# "CST 280 CHICAGO U INTRODUCTION ... B- 3.000 8.01"
# "CS 100 false UG Computers and Society A 3.000 12.00"
# "CST 280 CHICAGO CAMPUS U INTRODUCTION ... B- 3.000
8.01"
# "CST 280      CHICAGO      U  TITLE      A   3.000  12.00"
# "          CAMPUS
course_pattern = re.compile(
    r'^([A-Z]{1,5})\s+'          # Subject
    r'(\d{1,3}[A-Z]?)\s+'      # Course
number
    r'(?:'
        r'(?CHICAGO\s+(?CAMPUS\s+)?|ONLINE\s+
( ?CAMPUS\s+)?|false\s+)' # Campus
        r'(?U(?G)?|G)\s+'    # Level
    r')?'
    r'(.+?)\s+'              # Title
(greedy-lazy)
    r'([A-F][+-]?|P|W|WP|WF|I|IP|TCR)\s+' # Grade
    r'(\d+\.\d{3})\s+'        # Credit
hours
    r'(\d+\.\d{2})',         #
Quality points
    re.MULTILINE
)

for match in course_pattern.finditer(block):
    title = match.group(3).strip()
    # Filter out header/noise lines
    if title in ('Title', 'Subject') or 'Attempt' in
title:
        continue
    # Clean up multi-line campus text that may have bled
into title
    title = re.sub(r'^CAMPUS\s+', '', title)
    classes.append({
        'course': f"{match.group(1)} {match.group(2)}",
        'title': title,
        'grade': match.group(4).strip(),
        'credits': float(match.group(5)),
        'term': term_name,
    })

# — 8. Handle multi-line titles (CAMPUS wrapping)
# Roosevelt transcripts often wrap:
# "CST 280 CHICAGO U INTRODUCTION TO B- 3.000 8.01"
# "CAMPUS ALGORITHMS"
# The second line "CAMPUS ALGORITHMS" means campus label bled onto
next line.
# We already handle this via regex, but some titles may be

```

```

truncated.
    # Let's do a second pass on the raw text to catch wrapped titles.
    lines = full_text.split('\n')
    for i, cls in enumerate(classes):
        if cls['grade'] == "In Progress" or cls['term'] == "Transfer":
            continue
        # Find where this course appears in full_text
        course_parts = cls['course'].split()
        subj, num = course_parts[0], course_parts[1]
        # Look for the course line in full text, then check the next
line
        pattern = re.compile(
            rf'^{re.escape(subj)}\s+{re.escape(num)}\s+.{+
{re.escape(cls["grade"])}\s+',
            re.MULTILINE
        )
        match = pattern.search(full_text)
        if match:
            line_end = full_text.find('\n', match.start())
            if line_end != -1 and line_end + 1 < len(full_text):
                next_line = full_text[line_end + 1:
full_text.find('\n', line_end + 1)]
                next_line = next_line.strip()
                # If next line starts with "CAMPUS " and has more
text,
                # that text is the continuation of the title
                campus_cont = re.match(r'^CAMPUS\s+(.+)', next_line)
                if campus_cont:
                    continuation = campus_cont.group(1).strip()
                    # Don't append if it looks like a header or new
course
                    if not re.match(r'^[A-Z]{2,5}\s+\d',
continuation):
                        cls['title'] = cls['title'] + " " +
continuation

    return {
        'name': name,
        'institution': institution,
        'major': major,
        'minor': minor,
        'year': year,
        'classes': classes,
    }

def print_transcript_summary(data):
    """Pretty-print the extracted transcript data."""
    print("=" * 64)
    print(f"  Institution: {data.get('institution', 'N/A')}")

```

```

print(f"  Name:          {data['name']}")
print(f"  Major:         {data['major']}")
if data.get('minor'):
    print(f"  Minor:          {data['minor']}")
print(f"  Year:           {data['year']}")
print("=" * 64)

total_credits = 0
# Group by term
terms = {}
for c in data['classes']:
    terms.setdefault(c['term'], []).append(c)

for term, courses in terms.items():
    print(f"\n — {term} —")
    for c in courses:
        print(f"      {c['course']:<12} {c['title']:<40}
{c['grade']:<12} {c['credits']:.1f} cr")
        total_credits += c['credits']

print(f"\n{'=' * 64}")
print(f"  Total Classes: {len(data['classes'])}")
print(f"  Total Credits: {total_credits:.1f}")
print("=" * 64)

if __name__ == "__main__":
    import sys
    import glob

    # Accept a PDF path as argument, or process all PDFs in current
    directory
    if len(sys.argv) > 1:
        pdf_files = sys.argv[1:]
    else:
        pdf_files = glob.glob("*.pdf") + glob.glob("**/*.pdf",
recursive=True)

    if not pdf_files:
        print("Usage: python pdf_reader.py <transcript.pdf>
[transcript2.pdf ...]")
        print("  Or run in a directory containing PDF files.")
        sys.exit(1)

    for pdf_path in pdf_files:
        print(f"\n{'█' * 64}")
        print(f"  Processing: {pdf_path}")
        print(f"{'█' * 64}")
        try:
            data = extract_transcript_data(pdf_path)

```

```

        print_transcript_summary(data)
    except Exception as e:
        print(f"    △ Error processing {pdf_path}: {e}")

```

```

=====
=====
FILE: api
=====
=====

```

New Files We Produced

Only the new files created during this build are listed below.

Backend / API (New)

- `backend/app/main.py` - Python (FastAPI routes)
- `backend/app/config.py` - Python (environment/config settings)
- `backend/app/services.py` - Python (transcript matching service layer)
- `backend/app/ml\_service.py` - Python (ML integration service layer)
- `backend/app/schemas.py` - Python (Pydantic request/response schemas)
- `backend/run\_api.py` - Python (API server entrypoint)
- `backend/requirements.txt` - Text (Python dependencies)
- `backend/.env.example` - Env file (environment variable template)

Frontend (New)

- `frontend/src/App.jsx` - JavaScript React (main UI page logic)
- `frontend/src/main.jsx` - JavaScript React (frontend entrypoint)
- `frontend/src/components/MatchTable.jsx` - JavaScript React (results table component)
- `frontend/src/styles.css` - CSS (UI styling)
- `frontend/index.html` - HTML (root page shell)
- `frontend/vite.config.js` - JavaScript (Vite build/dev config)
- `frontend/package.json` - JSON (Node scripts and dependencies)
- `frontend/.env.example` - Env file (frontend variable template)

```

=====
=====
FILE: backend/.env.example
=====
=====

```

```

APP_ENV=dev
CSV_ROOT=../csv_exports
MATCHING_RULES_PATH=../config/matching_rules.json

```

Add your provider key later:

OPENAI_API_KEY=

=====

FILE: backend/app/config.py

=====

```
from __future__ import annotations
```

```
import os
from dataclasses import dataclass
from pathlib import Path
```

```
from dotenv import load_dotenv
```

```
_backend_dir = Path(__file__).resolve().parent.parent
_project_dir = _backend_dir.parent
load_dotenv(_backend_dir / ".env")
load_dotenv(_project_dir / ".env", override=False)
```

```
@dataclass
class Settings:
    csv_root: Path
    rules_path: Path
    openai_api_key: str | None
    app_env: str
```

```
def get_settings() -> Settings:
    project_root = _project_dir
    csv_root = Path(os.getenv("CSV_ROOT", project_root /
"csv_exports")).resolve()
    rules_path = Path(
        os.getenv("MATCHING_RULES_PATH", project_root / "config" /
"matching_rules.json")
    ).resolve()
    openai_api_key = os.getenv("OPENAI_API_KEY")
    app_env = os.getenv("APP_ENV", "dev")
    return Settings(
        csv_root=csv_root,
        rules_path=rules_path,
        openai_api_key=openai_api_key,
        app_env=app_env,
    )
```

=====

FILE: backend/app/main.py

```
=====
=====
```

```
from __future__ import annotations

from fastapi import FastAPI, File, HTTPException, Query, UploadFile
from fastapi.middleware.cors import CORSMiddleware

from .config import get_settings
from .ml_service import ml_analyze_kus, ml_predict, ml_predict_nlp,
ml_status, ml_train, ml_train_nlp
from .schemas import (
    AiExplainRequest,
    MLNLPPredictRequest,
    MLNLPTTrainRequest,
    MLPredictRequest,
    MLTrainRequest,
)
from .services import explain_match_stub, list_courses,
list_institutions, match_from_pdf_bytes

app = FastAPI(title="Transfer Credit Match API", version="1.0.0")
settings = get_settings()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/health")
def health() -> dict:
    return {"status": "ok", "env": settings.app_env}

@app.get("/api/institutions")
def institutions() -> list[dict]:
    return list_institutions(settings)

@app.get("/api/courses")
def courses(institution_id: int | None = Query(default=None)) ->
list[dict]:
    return list_courses(settings, institution_id=institution_id)

@app.post("/api/match/transcript")
```

```

async def match_transcript(
    target_institution_id: int = Query(...),
    top_k: int = Query(default=3, ge=1, le=10),
    threshold: float = Query(default=0.55, ge=0.0, le=1.0),
    pdf_file: UploadFile = File(...),
) -> dict:
    if not pdf_file.filename or not
pdf_file.filename.lower().endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF
transcript uploads are supported.")
    content = await pdf_file.read()
    if not content:
        raise HTTPException(status_code=400, detail="Uploaded file is
empty.")
    try:
        return match_from_pdf_bytes(
            settings=settings,
            pdf_bytes=content,
            filename=pdf_file.filename,
            target_institution_id=target_institution_id,
            top_k=top_k,
            threshold=threshold,
        )
    except Exception as exc: # noqa: BLE001
        raise HTTPException(status_code=500, detail=f"Failed to
process transcript: {exc}") from exc

@app.post("/api/ai/explain")
def explain(payload: AiExplainRequest) -> dict:
    return explain_match_stub(settings, payload.model_dump())

@app.get("/api/ml/status")
def model_status() -> dict:
    try:
        return ml_status()
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc

@app.post("/api/ml/train")
def model_train(payload: MLTrainRequest) -> dict:
    try:
        return ml_train(
            samples=payload.samples,
            epochs=payload.epochs,
            learning_rate=payload.learning_rate,
            patience=payload.patience,
        )
    
```

```

        except RuntimeError as exc:
            raise HTTPException(status_code=503, detail=str(exc)) from exc

@app.post("/api/ml/predict")
def model_predict(payload: MLPredictRequest) -> dict:
    try:
        if payload.source is not None and payload.target is not None:
            return ml_predict(source=payload.source,
target=payload.target)
        if payload.features is not None:
            return ml_predict(features=payload.features)
        raise HTTPException(status_code=400, detail="Provide
source+target or features.")
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc

@app.post("/api/ml/train-nlp")
def model_train_nlp(payload: MLNLPTTrainRequest) -> dict:
    try:
        return ml_train_nlp(
            csv_dir=payload.csv_dir,
            epochs=payload.epochs,
            learning_rate=payload.learning_rate,
            patience=payload.patience,
        )
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc

@app.post("/api/ml/predict-nlp")
def model_predict_nlp(payload: MLNLPPredictRequest) -> dict:
    try:
        return ml_predict_nlp(source=payload.source,
target=payload.target, csv_dir=payload.csv_dir)
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc

@app.post("/api/ml/analyze-kus")
def model_analyze_kus(payload: MLNLPPredictRequest) -> dict:
    try:
        return ml_analyze_kus(source=payload.source,
target=payload.target, csv_dir=payload.csv_dir)
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc

```

```

=====
=====

```

FILE: backend/app/ml_service.py

=====

```
from __future__ import annotations
```

```
import sys
from pathlib import Path
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
```

```
_tm_module = None
_tm_import_error: str | None = None
```

```
def _get_tm():
    """Lazy import so core API works without optional torch/sklearn ML
    stack."""
```

```
    global _tm_module, _tm_import_error
```

```
    if _tm_module is not None:
```

```
        return _tm_module
```

```
    if _tm_import_error is not None:
```

```
        raise RuntimeError(_tm_import_error)
```

```
    try:
```

```
        from ml import transfer_matcher_full as tm # noqa: E402
```

```
        _tm_module = tm
```

```
        return tm
```

```
    except ImportError as exc:
```

```
        _tm_import_error = (
```

```
            "ML routes need extra packages. From the backend folder
```

```
run: "
```

```
        f"pip install -r requirements.txt ({exc})"
```

```
    )
```

```
    raise RuntimeError(_tm_import_error) from exc
```

```
def ml_train(samples=None, epochs: int = 300, learning_rate: float =
0.001, patience: int = 50):
```

```
    tm = _get_tm()
```

```
    return tm.train_model(samples=samples, epochs=epochs,
```

```
    learning_rate=learning_rate, patience=patience)
```

```
def ml_predict(source=None, target=None, features=None):
```

```
    tm = _get_tm()
```

```
    return tm.predict(source=source, target=target, features=features)
```

```

def ml_status():
    tm = _get_tm()
    return tm.get_status()

def ml_train_nlp(csv_dir: str, epochs: int = 300, learning_rate: float
= 0.001, patience: int = 50):
    tm = _get_tm()
    return tm.train_nlp_model(csv_dir=csv_dir, epochs=epochs,
learning_rate=learning_rate, patience=patience)

def ml_predict_nlp(source: int, target: int, csv_dir: str):
    tm = _get_tm()
    return tm.predict_nlp(src_course_id=source, tgt_course_id=target,
csv_dir=csv_dir)

def ml_analyze_kus(source: int, target: int, csv_dir: str):
    tm = _get_tm()
    return tm.analyze_kus(src_course_id=source, tgt_course_id=target,
csv_dir=csv_dir)

=====
=====
FILE: backend/app/schemas.py
=====
=====

from __future__ import annotations

from pydantic import BaseModel, Field

class MatchRequest(BaseModel):
    target_institution_id: int
    top_k: int = Field(default=3, ge=1, le=10)
    threshold: float = Field(default=0.55, ge=0.0, le=1.0)

class AiExplainRequest(BaseModel):
    source_course_name: str
    target_course_name: str
    score: float
    decision: str

class MLTrainRequest(BaseModel):
    samples: list | None = None

```

```
epochs: int = 300
learning_rate: float = 0.001
patience: int = 50
```

```
class MLPredictRequest(BaseModel):
    source: list[float] | None = None
    target: list[float] | None = None
    features: list[float] | None = None
```

```
class MLNLPTTrainRequest(BaseModel):
    csv_dir: str = "csv_exports"
    epochs: int = 300
    learning_rate: float = 0.001
    patience: int = 50
```

```
class MLNLPPredictRequest(BaseModel):
    source: int
    target: int
    csv_dir: str = "csv_exports"
```

```
=====
=====
FILE: backend/app/services.py
=====
=====
```

```
from __future__ import annotations
```

```
import tempfile
from pathlib import Path
import sys
```

```
import pandas as pd
```

```
# Ensure project-root modules are importable when API runs from
backend/.
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
```

```
from TranscriptPDF_reader.pdf_reader import extract_transcript_data
from engine.config import load_matching_rules
from engine.data_loader import load_data_bundle
from engine.pipeline import match_transcript_to_institution
```

```
from .config import Settings
```

```

def list_institutions(settings: Settings) -> list[dict]:
    institutions = pd.read_csv(settings.csv_root / "institutions.csv")
    return institutions.to_dict(orient="records")

def list_courses(settings: Settings, institution_id: int | None =
None) -> list[dict]:
    courses = pd.read_csv(settings.csv_root / "courses.csv")
    if institution_id is not None:
        courses = courses[courses["institution_id"] ==
int(institution_id)]
    return courses.to_dict(orient="records")

def match_from_pdf_bytes(
    settings: Settings,
    pdf_bytes: bytes,
    filename: str,
    target_institution_id: int,
    top_k: int,
    threshold: float,
) -> dict:
    suffix = Path(filename).suffix or ".pdf"
    with tempfile.NamedTemporaryFile(suffix=suffix, delete=True) as
tmp:
        tmp.write(pdf_bytes)
        tmp.flush()
        transcript_data = extract_transcript_data(tmp.name)

    bundle = load_data_bundle(settings.csv_root)
    rules = load_matching_rules(settings.rules_path)
    matches = match_transcript_to_institution(
        transcript_data=transcript_data,
        target_institution_id=target_institution_id,
        data_bundle=bundle,
        top_k=top_k,
        rules=rules,
        approval_threshold=threshold,
    )
    return {
        "student": {
            "name": transcript_data.get("name"),
            "institution": transcript_data.get("institution"),
            "major": transcript_data.get("major"),
            "year": transcript_data.get("year"),
        },
        "total_parsed_classes": len(transcript_data.get("classes",
[])),
        "matches": matches.to_dict(orient="records"),

```

```

    }

def explain_match_stub(settings: Settings, payload: dict) -> dict:
    if settings.openai_api_key:
        return {
            "mode": "api-key-present",
            "explanation": (
                "API key detected. Connect this endpoint to your
preferred LLM provider "
                "when you are ready for AI-generated advisor
explanations."
            ),
        }
    return {
        "mode": "no-api-key",
        "explanation": (
            f"{payload['source_course_name']} vs
{payload['target_course_name']} "
            f"scored {payload['score']:.2f} and was classified as
{payload['decision']}. "
            "Add OPENAI_API_KEY in backend/.env to enable AI
explanation generation later."
        ),
    }

```

```

=====
=====
FILE: backend/requirements.txt
=====
=====

```

```

fastapi
uvicorn
python-multipart
pydantic
python-dotenv
pandas
pdfplumber
torch
scikit-learn
mysql-connector-python
requests

```

```

=====
=====
FILE: backend/run_api.py
=====
=====

```

```
from __future__ import annotations
```

```
import os
```

```
from pathlib import Path
```

```
import uvicorn
```

```
if __name__ == "__main__":
```

```
    backend_dir = Path(__file__).resolve().parent
```

```
    os.chdir(backend_dir)
```

```
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000,  
reload=True)
```

```
=====
```

```
=====
```

```
FILE: config/matching_rules.json
```

```
=====
```

```
=====
```

```
{  
  "global_default": {  
    "approval_threshold": 0.55,  
    "min_ku_overlap_ratio": 0.15,  
    "min_credit_ratio": 0.66,  
    "weights": {  
      "ku_overlap": 0.6,  
      "title_similarity": 0.2,  
      "credit_ratio": 0.1,  
      "syllabus_similarity": 0.1  
    }  
  },  
  "institution_rules": {  
    "4": {  
      "approval_threshold": 0.58,  
      "min_ku_overlap_ratio": 0.2  
    }  
  }  
}
```

```
=====
```

```
=====
```

```
FILE: csv_exports/admins.csv
```

```
=====
```

```
=====
```

```
admin_id,user_id,created_at
```

```
1,5,2026-02-16 20:25:19
```

```
2,22,2026-02-16 20:25:19
```

```
=====
```

=====

FILE: csv_exports/course_ku.csv

=====

=====

course_id,ku_id

1,1

2,2

3,3

4,4

23,5

23,6

13,7

24,7

5,8

14,8

25,8

12,9

24,10

23,11

13,12

12,13

13,14

23,15

25,15

17,16

17,17

12,18

23,19

11,20

13,20

24,21

12,22

14,23

13,24

23,25

23,26

24,26

=====

=====

FILE: csv_exports/course_match.csv

=====

=====

match_id,institution_from,institution_to,course_from,course_to,match_s
tatus,created_at

1,1,4,1,4,Pending,2026-02-16 20:25:26

=====

=====

FILE: csv_exports/course_syllabus.csv

=====

=====

course_id,syllabus_text

1,"Algorithms, data structures, complexity, recursion, arrays, linked lists, trees, graphs."
4,"Web technologies, HTML/CSS, JavaScript, basic databases, deployment and web security basics."
12,"Network protocols, routing, switching, subnetting, transport and application layers."
13,"Operating system design, process scheduling, memory management, file systems, virtualization."
17,"Database models, SQL queries, normalization, indexing, transactions, data integrity."
23,"Cybersecurity principles, threat modeling, vulnerabilities, cryptography, ethics, defense."
24,"Internet threats, secure communication, encryption, authentication, web application hardening."

=====

=====

FILE: csv_exports/courses.csv

=====

=====

course_id,institution_id,program_id,course_name,course_code,credits,created_at

1,1,1,Data Structures and Algorithms,CS101,4,2026-02-16 20:24:01
2,1,2,Network Security,CYB201,3,2026-02-16 20:24:01
3,4,3,IT Fundamentals,IT101,3,2026-02-16 20:24:01
4,4,4,Web Development,SD102,3,2026-02-16 20:24:01
5,1,9,Computer Science I,CSIA 150,4,2026-02-16 20:24:01
6,1,9,Intro to Prob. & Stats,CSIA 217,3,2026-02-16 20:24:01
7,1,9,Python Script Programming,CSIA 236,3,2026-02-16 20:24:01
8,1,9,Data Communications,CSIA 246,3,2026-02-16 20:24:01
9,1,9,Computer Science II,CSIA 250,4,2026-02-16 20:24:01
10,1,9,Open Source Communities,CSIA 255,3,2026-02-16 20:24:01
11,1,9,Computer Organization,CSIA 261,3,2026-02-16 20:24:01
12,1,9,Computer Networking,CSIA 301,3,2026-02-16 20:24:01
13,1,9,Operating Systems,CSIA 317,3,2026-02-16 20:24:01
14,1,9,UNIX and System Administration,CSIA 318,3,2026-02-16 20:24:01
15,1,9,Cyber Ops,CSIA 319,3,2026-02-16 20:24:01
16,1,9,Software Engineering,CSIA 327,3,2026-02-16 20:24:01
17,1,9,Database Systems,CSIA 333,3,2026-02-16 20:24:01
18,1,9,Ethical Hack & Countermeasures,CSIA 335,3,2026-02-16 20:24:01
19,1,9,Practical Computing with Data in Python,CSIA 336,3,2026-02-16 20:24:01
20,1,9,Network Design,CSIA 352,3,2026-02-16 20:24:01

21,1,9,Introduction to Programming,CSIA 354,3,2026-02-16 20:24:01
22,1,9,Applied Cryptography,CSIA 355,3,2026-02-16 20:24:01
23,1,9,Intro to Computer Security,CSIA 359,3,2026-02-16 20:24:01
24,1,9,Internet Security,CSIA 368,3,2026-02-16 20:24:01
25,1,9,Senior Project,CSIA 399,3,2026-02-16 20:24:01

=====

=====

FILE: csv_exports/directors.csv

=====

=====

director_id,user_id,institution_id,created_at
1,1,1,2026-02-16 20:22:36
2,2,4,2026-02-16 20:22:36
3,11,4,2026-02-16 20:22:36
4,18,2,2026-02-16 20:22:36
5,19,3,2026-02-16 20:22:36
6,10,1,2026-02-16 20:22:36

=====

=====

FILE: csv_exports/institutions.csv

=====

=====

institution_id,name,location,created_at
1,Roosevelt University,"Chicago, IL",2026-02-16 20:22:11
2,National Louis University,"Chicago, IL",2026-02-16 20:22:11
3,University of Chicago,"Chicago, IL",2026-02-16 20:22:11
4,Harold Washington College,"Chicago, IL",2026-02-16 20:22:11

=====

=====

FILE: csv_exports/knowledge_units.csv

=====

=====

ku_id,ku_name,ku_description,created_at
1,Programming Concepts,Covers fundamental programming logic and syntax,2026-02-16 20:24:21
2,Cybersecurity Basics,Introduction to security concepts and best practices,2026-02-16 20:24:21
3,Networking Fundamentals,Covers basic networking principles and protocols,2026-02-16 20:24:21
4,Database Management,Covers SQL and NoSQL database principles,2026-02-16 20:24:21
5,Cybersecurity Foundations,Covers fundamental concepts behind cybersecurity,2026-02-16 20:24:21
6,Cybersecurity Principles,Covers basic security design

fundamentals,2026-02-16 20:24:21

7,IT Systems Components,Covers components in an IT system and their roles in system operation,2026-02-16 20:24:21

8,Basic Scripting and Programming,"Covers simple script/program creation, algorithm implementation, and basic security practices in scripting/programming",2026-02-16 20:24:21

9,Basic Networking,"Covers how networks are built and operated, network analysis tools, and vulnerabilities",2026-02-16 20:24:21

10,Network Defense,"Covers defense of networks, basic tools/techniques to protect networks, and communication assets from cyber threats",2026-02-16 20:24:21

11,Basic Cryptography,Covers how and where cryptography is used,2026-02-16 20:24:21

12,Operating Systems Concepts,"Covers roles of operating systems, basic functions, and services provided by operating systems",2026-02-16 20:24:21

13,Advanced Network Technology and Protocols,"Covers advanced networking concepts in technology, security, and communication",2026-02-16 20:24:21

14,Cloud Computing,"Covers technologies and services that enable cloud computing, types of cloud computing models, and security/legal issues with cloud computing",2026-02-16 20:24:21

15,Cybersecurity Ethics,Covers ethics in cyber contexts,2026-02-16 20:24:21

16,Database Management Systems,Covers skills to utilize database management systems,2026-02-16 20:24:21

17,Databases,"Covers how database systems are used, managed, and associated issues",2026-02-16 20:24:21

18,Digital Communications,Covers protocols and methodologies in modern digital communications systems,2026-02-16 20:24:21

19,Intrusion Detection/Prevention Systems,Covers skills related to detecting/analyzing vulnerabilities and risk mitigation,2026-02-16 20:24:21

20,Low Level Programming,Covers programming securely with low level programming languages,2026-02-16 20:24:21

21,Network Security Administration,Covers maintaining comprehensive enterprise security infrastructure,2026-02-16 20:24:21

22,Network Technology and Protocols,"Covers common network protocols, interaction between network components, and network evolution",2026-02-16 20:24:21

23,Operating Systems Administration,Covers basic operations involved in system administration of operating systems,2026-02-16 20:24:21

24,Operating Systems Theory,"Covers implementation of operating system concepts, components, and interfaces",2026-02-16 20:24:21

25,Vulnerability Analysis,Covers system vulnerabilities,2026-02-16 20:24:21

26,Web Application Security,"Covers technology, tools, and practices associated with web applications",2026-02-16 20:24:21

=====

=====

FILE: csv_exports/match_history.csv

=====

=====

log_id,match_id,changed_by,change_type,change_date

=====

=====

FILE: csv_exports/programs.csv

=====

=====

program_id,institution_id,program_name,created_at

1,1,Computer Science,2026-02-16 20:22:51

2,1,Cybersecurity,2026-02-16 20:22:51

3,4,Information Technology,2026-02-16 20:22:51

4,4,Software Development,2026-02-16 20:22:51

5,2,Computer Forensics,2026-02-16 20:22:51

6,2,Computer Science,2026-02-16 20:22:51

7,3,Cloud Computing,2026-02-16 20:22:51

8,3,Information Technology,2026-02-16 20:22:51

9,1,Cyber Security and Information Assurance,2026-02-16 20:22:51

=====

=====

FILE: csv_exports/students.csv

=====

=====

student_id,user_id,institution_id,program_id,created_at

1,3,1,1,2026-02-16 20:25:01

2,4,4,4,2026-02-16 20:25:01

3,6,1,1,2026-02-16 20:25:01

4,7,4,3,2026-02-16 20:25:01

5,8,1,2,2026-02-16 20:25:01

6,9,4,4,2026-02-16 20:25:01

=====

=====

FILE: csv_exports/transfer_requests.csv

=====

=====

request_id,student_id,course_from,course_to,institution_from,institution_to,status,request_date

1,1,1,4,1,4,Pending,2026-02-16 20:25:34

=====

=====

FILE: csv_exports/users.csv

=====

```
user_id,name,email,password_hash,role,created_at
1,Alice
Johnson,alice.johnson@roosevelt.edu,hashed_password_1,director,2026-02-16 20:22:20
2,Bob Smith,bob.smith@hwc.edu,hashed_password_2,director,2026-02-16 20:22:20
3,Charlie
Davis,charlie.davis@roosevelt.edu,hashed_password_3,student,2026-02-16 20:22:20
4,Diana Lee,diana.lee@hwc.edu,hashed_password_4,student,2026-02-16 20:22:20
5,Eve
Thompson,eve.thompson@admin.com,hashed_password_5,admin,2026-02-16 20:22:20
6,Sam
Harper,sam.harper@roosevelt.edu,hashed_password_6,student,2026-02-16 20:22:20
7,Liam Carter,liam.carter@hwc.edu,hashed_password_7,student,2026-02-16 20:22:20
8,Elena
Brooks,elena.brooks@roosevelt.edu,hashed_password_8,student,2026-02-16 20:22:20
9,Mason Reed,mason.reed@hwc.edu,hashed_password_9,student,2026-02-16 20:22:20
10,Sophia
Bennett,sophia.bennett@roosevelt.edu,hashed_password_10,director,2026-02-16 20:22:20
11,Ethan
Hayes,ethan.hayes@hwc.edu,hashed_password_11,director,2026-02-16 20:22:20
12,Amara
Santos,amara.santos@NLU.edu,hashed_password_13,student,2026-02-16 20:22:20
13,Kai
Mendoza,kai.mendoza@NLU.edu,hashed_password_14,student,2026-02-16 20:22:20
14,Elliot
Harper,elliott.harper@NLU.edu,hashed_password_15,student,2026-02-16 20:22:20
15,Arjun
Patel,arjun.patel@uofc.edu,hashed_password_16,student,2026-02-16 20:22:20
16,Amina
Hassan,amina.hassan@uofc.edu,hashed_password_17,student,2026-02-16 20:22:20
17,Carter
```

```
Hayes,carter.hayes@uofc.edu,hashed_password_18,student,2026-02-16
20:22:20
18,Morgan
Blake,morgan.blake@NLU.edu,hashed_password_19,director,2026-02-16
20:22:20
19,Jordan
Grant,jordan.grant@uofc.edu,hashed_password_20,director,2026-02-16
20:22:20
20,Mike Wells,mike.wells@admin.com,hashed_password_21,admin,2026-02-16
20:22:20
21,Susan
Bennett,susan.bennett@admin.com,hashed_password_22,admin,2026-02-16
20:22:20
22,Chloe
Dawson,chloe.dawson@admin.com,hashed_password_12,admin,2026-02-16
20:22:20
```

```
=====
=====
FILE: engine/__init__.py
=====
=====
```

```
"""Transfer Credit Match data-science engine."""
```

```
from .data_loader import load_data_bundle
from .matcher import compute_course_match
from .pipeline import match_transcript_to_institution
from .workflow import apply_match_decision,
create_transfer_request_and_match
```

```
__all__ = [
    "load_data_bundle",
    "compute_course_match",
    "match_transcript_to_institution",
    "create_transfer_request_and_match",
    "apply_match_decision",
]
```

```
=====
=====
FILE: engine/config.py
=====
=====
```

```
from __future__ import annotations

import json
from pathlib import Path
```

```

DEFAULT_RULES = {
    "global_default": {
        "approval_threshold": 0.55,
        "min_ku_overlap_ratio": 0.15,
        "min_credit_ratio": 0.66,
        "weights": {
            "ku_overlap": 0.60,
            "title_similarity": 0.20,
            "credit_ratio": 0.10,
            "syllabus_similarity": 0.10,
        },
    },
    "institution_rules": {},
}

```

```

def load_matching_rules(path: str | Path | None) -> dict:
    if path is None:
        return DEFAULT_RULES
    rules_path = Path(path)
    if not rules_path.exists():
        return DEFAULT_RULES
    with rules_path.open("r", encoding="utf-8") as f:
        loaded = json.load(f)
    merged = DEFAULT_RULES.copy()
    merged.update(loaded)
    return merged

```

```

def resolve_rule(rules: dict, institution_id: int) -> dict:
    specific = rules.get("institution_rules",
    {}).get(str(institution_id), {})
    base = dict(rules.get("global_default", {}))
    base_weights = dict(base.get("weights", {}))
    base_weights.update(specific.get("weights", {}))
    base["weights"] = base_weights
    for key, value in specific.items():
        if key != "weights":
            base[key] = value
    return base

```

```

=====
=====
FILE: engine/data_loader.py
=====
=====

```

```

from __future__ import annotations

```

```
from dataclasses import dataclass
from pathlib import Path
```

```
import pandas as pd
```

```
@dataclass
class DataBundle:
    courses: pd.DataFrame
    course_ku: pd.DataFrame
    knowledge_units: pd.DataFrame
    institutions: pd.DataFrame
    programs: pd.DataFrame
    transfer_requests: pd.DataFrame
    students: pd.DataFrame
    course_syllabus: pd.DataFrame | None = None
```

```
def _read_csv(root: Path, name: str) -> pd.DataFrame:
    path = root / name
    if not path.exists():
        raise FileNotFoundError(f"Required dataset is missing:
{path}")
    return pd.read_csv(path)
```

```
def load_data_bundle(csv_root: str | Path) -> DataBundle:
    csv_root_path = Path(csv_root).resolve()
    syllabus_path = csv_root_path / "course_syllabus.csv"
    syllabus_df = pd.read_csv(syllabus_path) if syllabus_path.exists()
    else None
    return DataBundle(
        courses=_read_csv(csv_root_path, "courses.csv"),
        course_ku=_read_csv(csv_root_path, "course_ku.csv"),
        knowledge_units=_read_csv(csv_root_path,
"knowledge_units.csv"),
        institutions=_read_csv(csv_root_path, "institutions.csv"),
        programs=_read_csv(csv_root_path, "programs.csv"),
        transfer_requests=_read_csv(csv_root_path,
"transfer_requests.csv"),
        students=_read_csv(csv_root_path, "students.csv"),
        course_syllabus=syllabus_df,
    )
```

```
=====
=====
FILE: engine/matcher.py
=====
=====
```

```
from __future__ import annotations

from dataclasses import dataclass
from difflib import SequenceMatcher
from typing import Iterable

from .text_similarity import cosine_token_similarity,
jaccard_similarity
```

```
@dataclass
class MatchResult:
    source_course_code: str
    source_course_name: str
    target_course_code: str
    target_course_name: str
    source_credits: float
    target_credits: float
    ku_overlap_count: int
    ku_overlap_ratio: float
    title_similarity: float
    syllabus_similarity: float
    score: float
    decision: str
```

```
def _safe_ratio(a: float, b: float) -> float:
    if a <= 0 or b <= 0:
        return 0.0
    return min(a, b) / max(a, b)
```

```
def _title_similarity(a: str, b: str) -> float:
    return SequenceMatcher(None, a.lower().strip(),
b.lower().strip()).ratio()
```

```
def compute_course_match(
    source_course: dict,
    target_course: dict,
    source_ku_ids: Iterable[int],
    target_ku_ids: Iterable[int],
    source_syllabus: str = "",
    target_syllabus: str = "",
    rule: dict | None = None,
    approval_threshold: float = 0.55,
) -> MatchResult:
    source_set = set(source_ku_ids)
    target_set = set(target_ku_ids)
    overlap = source_set & target_set
```

```

        union_size = len(source_set | target_set)
        overlap_ratio = len(overlap) / union_size if union_size else 0.0
        title_sim = _title_similarity(source_course["course_name"],
target_course["course_name"])
        credit_ratio = _safe_ratio(float(source_course["credits"]),
float(target_course["credits"]))
        syllabus_sim = (
            0.5 * cosine_token_similarity(source_syllabus,
target_syllabus)
            + 0.5 * jaccard_similarity(source_syllabus, target_syllabus)
        )

        effective_rule = rule or {}
        weights = effective_rule.get(
            "weights",
            {"ku_overlap": 0.60, "title_similarity": 0.20, "credit_ratio":
0.10, "syllabus_similarity": 0.10},
        )
        min_ku_overlap = float(effective_rule.get("min_ku_overlap_ratio",
0.0))
        min_credit_ratio = float(effective_rule.get("min_credit_ratio",
0.0))
        effective_threshold =
float(effective_rule.get("approval_threshold", approval_threshold))

        score = (
            float(weights.get("ku_overlap", 0.0)) * overlap_ratio
            + float(weights.get("title_similarity", 0.0)) * title_sim
            + float(weights.get("credit_ratio", 0.0)) * credit_ratio
            + float(weights.get("syllabus_similarity", 0.0)) *
syllabus_sim
        )

        if overlap_ratio == 0:
            decision = "Not Equivalent"
        elif overlap_ratio < min_ku_overlap or credit_ratio <
min_credit_ratio:
            decision = "Review"
        elif score >= effective_threshold:
            decision = "Equivalent"
        else:
            decision = "Review"

        return MatchResult(
            source_course_code=source_course["course_code"],
            source_course_name=source_course["course_name"],
            target_course_code=target_course["course_code"],
            target_course_name=target_course["course_name"],
            source_credits=float(source_course["credits"]),

```

```

        target_credits=float(target_course["credits"]),
        ku_overlap_count=len(overlap),
        ku_overlap_ratio=round(overlap_ratio, 4),
        title_similarity=round(title_sim, 4),
        syllabus_similarity=round(syllabus_sim, 4),
        score=round(score, 4),
        decision=decision,
    )

```

```

=====
=====
FILE: engine/pipeline.py
=====
=====

```

```

from __future__ import annotations

```

```

from dataclasses import asdict

```

```

import pandas as pd

```

```

from .config import resolve_rule

```

```

from .data_loader import DataBundle

```

```

from .matcher import compute_course_match

```

```

def _index_course_ku(course_ku_df: pd.DataFrame) -> dict[int,
set[int]]:
    grouped = course_ku_df.groupby("course_id")["ku_id"].apply(set)
    return grouped.to_dict()

```

```

def _normalize_transcript_course_code(value: str) -> str:
    return " ".join(str(value).strip().split())

```

```

def match_transcript_to_institution(
    transcript_data: dict,
    target_institution_id: int,
    data_bundle: DataBundle,
    top_k: int = 3,
    rules: dict | None = None,
    approval_threshold: float = 0.55,
) -> pd.DataFrame:
    courses = data_bundle.courses.copy()
    ku_index = _index_course_ku(data_bundle.course_ku)

    target_courses = courses[courses["institution_id"] ==
int(target_institution_id)]
    syllabus_index = {}

```

```

        if data_bundle.course_syllabus is not None and not
data_bundle.course_syllabus.empty:
            syllabus_index = dict(
                zip(
                    data_bundle.course_syllabus["course_id"].astype(int),
data_bundle.course_syllabus["syllabus_text"].astype(str),
                )
            )
            rule = resolve_rule(rules or {"global_default": {}},
int(target_institution_id))

            if target_courses.empty:
                raise ValueError(f"No target courses found for
institution_id={target_institution_id}")

            transcript_classes = transcript_data.get("classes", [])
            if not transcript_classes:
                return pd.DataFrame()

            transcript_index = {
                _normalize_transcript_course_code(item["course"]): item for
item in transcript_classes
            }

            matches = []
            for source_code, source_item in transcript_index.items():
                source_lookup = courses[courses["course_code"] == source_code]
                if source_lookup.empty:
                    continue
                source_course = source_lookup.iloc[0].to_dict()
                source_ku = ku_index.get(int(source_course["course_id"]),
set())

                scored_candidates = []
                for _, target_course in target_courses.iterrows():
                    target_dict = target_course.to_dict()
                    target_ku = ku_index.get(int(target_dict["course_id"]),
set())

                    result = compute_course_match(
                        source_course=source_course,
                        target_course=target_dict,
                        source_ku_ids=source_ku,
                        target_ku_ids=target_ku,

source_syllabus=syllabus_index.get(int(source_course["course_id"]),
""),

target_syllabus=syllabus_index.get(int(target_dict["course_id"]), ""),
rule=rule,

```

```

        approval_threshold=approval_threshold,
    )
    scored_candidates.append(result)

    for result in sorted(scored_candidates, key=lambda x: x.score,
reverse=True)[:top_k]:
        row = asdict(result)
        row["source_term"] = source_item.get("term", "Unknown")
        row["source_grade"] = source_item.get("grade", "Unknown")
        matches.append(row)

    return pd.DataFrame(matches).sort_values(
        by=["source_course_code", "score"], ascending=[True, False]
    )

```

```

=====
=====
FILE: engine/text_similarity.py
=====
=====

```

```

from __future__ import annotations

```

```

import re
from collections import Counter

```

```

TOKEN_PATTERN = re.compile(r"[a-zA-Z0-9]+")

```

```

def tokenize(text: str) -> list[str]:
    return TOKEN_PATTERN.findall((text or "").lower())

```

```

def jaccard_similarity(a: str, b: str) -> float:
    set_a = set(tokenize(a))
    set_b = set(tokenize(b))
    if not set_a and not set_b:
        return 0.0
    union = set_a | set_b
    if not union:
        return 0.0
    return len(set_a & set_b) / len(union)

```

```

def cosine_token_similarity(a: str, b: str) -> float:
    tok_a = Counter(tokenize(a))
    tok_b = Counter(tokenize(b))
    if not tok_a or not tok_b:
        return 0.0

```

```

        dot = sum(tok_a[token] * tok_b[token] for token in set(tok_a) &
set(tok_b))
        mag_a = sum(v * v for v in tok_a.values()) ** 0.5
        mag_b = sum(v * v for v in tok_b.values()) ** 0.5
        if mag_a == 0.0 or mag_b == 0.0:
            return 0.0
        return dot / (mag_a * mag_b)

```

```

=====
=====
FILE: engine/workflow.py
=====
=====

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass
from datetime import datetime
from pathlib import Path

```

```

import pandas as pd

```

```

def _now_str() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

```

def _csv_path(csv_root: str | Path, file_name: str) -> Path:
    return Path(csv_root).resolve() / file_name

```

```

def _next_id(df: pd.DataFrame, id_col: str) -> int:
    if df.empty:
        return 1
    return int(df[id_col].max()) + 1

```

```

@dataclass
class RequestResult:
    request_id: int
    match_id: int
    course_to: int
    status: str

```

```

def create_transfer_request_and_match(
    csv_root: str | Path,
    student_id: int,
    institution_from: int,

```

```

        institution_to: int,
        course_from: int,
        course_to: int,
    ) -> RequestResult:
        root = Path(csv_root).resolve()
        requests_path = _csv_path(root, "transfer_requests.csv")
        match_path = _csv_path(root, "course_match.csv")

        requests_df = pd.read_csv(requests_path)
        match_df = pd.read_csv(match_path)

        request_id = _next_id(requests_df, "request_id")
        match_id = _next_id(match_df, "match_id")
        now = _now_str()

        req_row = pd.DataFrame(
            [
                {
                    "request_id": request_id,
                    "student_id": int(student_id),
                    "course_from": int(course_from),
                    "course_to": int(course_to),
                    "institution_from": int(institution_from),
                    "institution_to": int(institution_to),
                    "status": "Pending",
                    "request_date": now,
                }
            ]
        )
        match_row = pd.DataFrame(
            [
                {
                    "match_id": match_id,
                    "institution_from": int(institution_from),
                    "institution_to": int(institution_to),
                    "course_from": int(course_from),
                    "course_to": int(course_to),
                    "match_status": "Pending",
                    "created_at": now,
                }
            ]
        )

        pd.concat([requests_df, req_row],
            ignore_index=True).to_csv(requests_path, index=False)
        pd.concat([match_df, match_row],
            ignore_index=True).to_csv(match_path, index=False)

        return RequestResult(
            request_id=request_id,

```

```

        match_id=match_id,
        course_to=int(course_to),
        status="Pending",
    )

def apply_match_decision(
    csv_root: str | Path,
    match_id: int,
    changed_by: int,
    decision: str,
) -> None:
    if decision not in {"Approved", "Denied"}:
        raise ValueError("decision must be one of: Approved, Denied")

    root = Path(csv_root).resolve()
    match_path = _csv_path(root, "course_match.csv")
    history_path = _csv_path(root, "match_history.csv")

    match_df = pd.read_csv(match_path)
    history_df = pd.read_csv(history_path)

    match_mask = match_df["match_id"] == int(match_id)
    if not match_mask.any():
        raise ValueError(f"match_id={match_id} not found")

    match_df.loc[match_mask, "match_status"] = decision
    match_df.to_csv(match_path, index=False)

    log_id = _next_id(history_df, "log_id")
    hist_row = pd.DataFrame(
        [
            {
                "log_id": log_id,
                "match_id": int(match_id),
                "changed_by": int(changed_by),
                "change_type": decision,
                "change_date": _now_str(),
            }
        ]
    )
    pd.concat([history_df, hist_row],
ignore_index=True).to_csv(history_path, index=False)

```

```

=====
=====
FILE: frontend/.env.example
=====
=====

```

```
# Leave empty for local dev (Vite proxies /api to the backend).
VITE_API_BASE_URL=
```

```
# Production example:
```

```
# VITE_API_BASE_URL=https://your-api-host.example.com
```

```
=====
=====
FILE: frontend/index.html
=====
=====
```

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Transfer Credit Match</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

```
=====
=====
FILE: frontend/package.json
=====
=====
```

```
{
  "name": "transfer-credit-match-frontend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^5.4.11"
  }
}
```

```
}
```

```
=====
=====
```

```
FILE: frontend/src/App.jsx
```

```
=====
=====
```

```
import { useEffect, useMemo, useState } from "react";
import MatchTable from "../components/MatchTable";
```

```
// Empty string uses Vite dev proxy (see vite.config.js). Set
VITE_API_BASE_URL for production.
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? "";
```

```
async function apiGet(path) {
  const res = await fetch(`${API_BASE_URL}${path}`);
  if (!res.ok) {
    throw new Error(`GET ${path} failed: ${res.status}`);
  }
  return res.json();
}
```

```
async function apiMatchTranscript({ file, targetInstitutionId, topK,
threshold }) {
```

```
  const formData = new FormData();
  formData.append("pdf_file", file);
```

```
  const url = new URL(`${API_BASE_URL}/api/match/transcript`);
  url.searchParams.set("target_institution_id",
String(targetInstitutionId));
  url.searchParams.set("top_k", String(topK));
  url.searchParams.set("threshold", String(threshold));
```

```
  const res = await fetch(url, {
    method: "POST",
    body: formData
  });
```

```
  if (!res.ok) {
    const msg = await res.text();
    throw new Error(msg || `Upload failed with status ${res.status}`);
  }
```

```
  return res.json();
}
```

```
async function apiExplain(row) {
```

```
  const res = await fetch(`${API_BASE_URL}/api/ai/explain`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
```

```

        source_course_name: row.source_course_name,
        target_course_name: row.target_course_name,
        score: row.score,
        decision: row.decision
    })
});
if (!res.ok) {
    throw new Error("Explanation request failed.");
}
return res.json();
}

export default function App() {
    const [institutions, setInstitutions] = useState([]);
    const [targetInstitutionId, setTargetInstitutionId] = useState("");
    const [file, setFile] = useState(null);
    const [topK, setTopK] = useState(3);
    const [threshold, setThreshold] = useState(0.55);
    const [loading, setLoading] = useState(false);
    const [result, setResult] = useState(null);
    const [error, setError] = useState("");
    const [aiExplanation, setAiExplanation] = useState("");

    useEffect(() => {
        apiGet("/api/institutions")
            .then((data) => {
                setInstitutions(data);
                if (data.length) {
                    setTargetInstitutionId(String(data[0].institution_id));
                }
            })
            .catch((err) => setError(err.message));
    }, []);

    const bestMatch = useMemo(() => {
        if (!result?.matches?.length) return null;
        return [...result.matches].sort((a, b) => b.score - a.score)[0];
    }, [result]);

    async function handleSubmit(e) {
        e.preventDefault();
        setError("");
        setAiExplanation("");
        if (!file) {
            setError("Please select a transcript PDF.");
            return;
        }
        if (!targetInstitutionId) {
            setError("Please select a destination institution.");
            return;
        }
    }

```

```

    }
    setLoading(true);
    try {
      const data = await apiMatchTranscript({
        file,
        targetInstitutionId,
        topK,
        threshold
      });
      setResult(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }
}

async function handleExplainBestMatch() {
  if (!bestMatch) return;
  setAiExplanation("Loading explanation...");
  try {
    const data = await apiExplain(bestMatch);
    setAiExplanation(data.explanation);
  } catch (err) {
    setAiExplanation(`Could not get explanation: ${err.message}`);
  }
}

return (
  <main className="container">
    <h1>Transfer Credit Match</h1>
    <p className="subtitle">
      Upload transcript PDF -> parse courses -> compare with KU
      mappings -> review likely equivalencies.
    </p>

    <form className="card" onSubmit={handleSubmit}>
      <div className="grid">
        <label>
          Destination Institution
          <select
            value={targetInstitutionId}
            onChange={(e) => setTargetInstitutionId(e.target.value)}
          >
            {institutions.map((inst) => (
              <option key={inst.institution_id}
value={inst.institution_id}>
                {inst.name}
              </option>
            ))}
          </select>
        </label>
      </div>
    </form>
  </main>
)

```

```

        </select>
    </label>
    <label>
        Top K Matches per Course
        <input
            type="number"
            min="1"
            max="10"
            value={topK}
            onChange={(e) => setTopK(Number(e.target.value))}
        />
    </label>
    <label>
        Auto-Equivalent Threshold
        <input
            type="number"
            step="0.01"
            min="0"
            max="1"
            value={threshold}
            onChange={(e) => setThreshold(Number(e.target.value))}
        />
    </label>
</div>
<label>
    Transcript PDF
    <input
        type="file"
        accept="application/pdf"
        onChange={(e) => setFile(e.target.files?.[0] || null)}
    />
</label>
<button type="submit" disabled={loading}>
    {loading ? "Processing..." : "Run Transfer Match"}
</button>
</form>

{error ? <p className="error">{error}</p> : null}

{result?.student ? (
    <section className="card">
        <h2>Transcript Summary</h2>
        <p><strong>Name:</strong> {result.student.name}</p>
        <p><strong>Institution:</strong>
{result.student.institution}</p>
        <p><strong>Major:</strong> {result.student.major}</p>
        <p><strong>Year:</strong> {result.student.year}</p>
        <p><strong>Parsed Classes:</strong>
{result.total_parsed_classes}</p>
    </section>

```

```

    ) : null}

    <section className="card">
      <h2>Match Results</h2>
      <MatchTable rows={result?.matches || []} />
    </section>

    <section className="card">
      <h2>Advisor Explanation (API key optional)</h2>
      <button onClick={handleExplainBestMatch} disabled={!bestMatch}
>
        Explain Best Match
      </button>
      <p className="mono">{aiExplanation || "No explanation
generated yet."}</p>
    </section>
  </main>
);
}

```

```

=====
=====
FILE: frontend/src/components/MatchTable.jsx
=====
=====

```

```

export default function MatchTable({ rows }) {
  if (!rows?.length) {
    return <p className="empty">No matches yet. Upload a transcript
PDF to begin.</p>;
  }

  return (
    <div className="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Source Course</th>
            <th>Target Course</th>
            <th>KU Overlap</th>
            <th>Score</th>
            <th>Decision</th>
          </tr>
        </thead>
        <tbody>
          {rows.map((row, idx) => (
            <tr key={` ${row.source_course_code}-${
{row.target_course_code}-${idx}`}>
              <td>
                <strong>{row.source_course_code}</strong>

```

```

        <div>{row.source_course_name}</div>
      </td>
      <td>
        <strong>{row.target_course_code}</strong>
        <div>{row.target_course_name}</div>
      </td>
      <td>
        {row.ku_overlap_count} ({row.ku_overlap_ratio})
      </td>
      <td>{row.score}</td>
      <td>{row.decision}</td>
    </tr>
  )}
</tbody>
</table>
</div>
);
}

```

```

=====
=====
FILE: frontend/src/main.jsx
=====
=====

```

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./styles.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

```

=====
=====
FILE: frontend/src/styles.css
=====
=====

```

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto,
  sans-serif;
}

```

```
    background: #f4f7fb;
    color: #1a202c;
}

.container {
    max-width: 1100px;
    margin: 0 auto;
    padding: 24px;
}

.subtitle {
    color: #4a5568;
}

.card {
    background: #fff;
    border-radius: 12px;
    padding: 16px;
    margin: 16px 0;
    box-shadow: 0 4px 14px rgba(15, 23, 42, 0.08);
}

.grid {
    display: grid;
    gap: 12px;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}

label {
    display: flex;
    flex-direction: column;
    gap: 6px;
    font-weight: 600;
    margin: 10px 0;
}

input,
select,
button {
    border: 1px solid #cbd5e0;
    border-radius: 8px;
    padding: 10px;
    font: inherit;
}

button {
    background: #2563eb;
    color: #fff;
    border: 0;
    cursor: pointer;
}
```

```

    font-weight: 600;
}

button:disabled {
    opacity: 0.5;
    cursor: not-allowed;
}

.error {
    color: #c53030;
    font-weight: 700;
}

.table-wrap {
    overflow: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th,
td {
    border: 1px solid #e2e8f0;
    text-align: left;
    padding: 10px;
    vertical-align: top;
}

thead {
    background: #edf2f7;
}

.empty {
    color: #718096;
}

.mono {
    font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas,
monospace;
}

```

```

=====
=====
FILE: frontend/vite.config.js
=====
=====

```

```
import { defineConfig } from "vite";
```

```
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      "/api": {
        target: "http://127.0.0.1:8000",
        changeOrigin: true
      },
      "/health": {
        target: "http://127.0.0.1:8000",
        changeOrigin: true
      }
    }
  }
});
```

```
=====
=====
FILE: ml/__init__.py
=====
=====
```

```
"""ML package for optional neural-network matcher (requires torch)."""
```

```
=====
=====
FILE: ml/transfer_matcher_full.py
=====
=====
```

```
#!/usr/bin/env python3
"""
```

```
Transfer Course Matching System – v3 (Sprint III – Blocking Strategy)
```

```
=====
Integrated into Transfer Credit Match project.
```

```
"""
```

```
import sys, os, json, re, argparse, datetime
from collections import defaultdict
```

```
import torch
import torch.nn as nn
import torch.optim as optim
```

```
MODEL_PATH = os.path.join(os.path.dirname(__file__),
"model_weights.pt")
MODEL_META_PATH = os.path.join(os.path.dirname(__file__),
```

```
"model_meta.json")
```

```
NUM_SUBJECTS = 6  
FEATURE_SIZE = 19
```

```
def build_feature_vector(src, tgt):  
    sc, sl, ss, slab = float(src[0]), int(src[1]), int(src[2]),  
float(src[3])  
    tc, tl, ts = float(tgt[0]), int(tgt[1]), int(tgt[2])  
    src_oh = [1.0 if i == ss else 0.0 for i in range(NUM_SUBJECTS)]  
    tgt_oh = [1.0 if i == ts else 0.0 for i in range(NUM_SUBJECTS)]  
    return [  
        1.0 if ss == ts else 0.0,  
        float(sl - tl),  
        1.0 if sl >= tl else 0.0,  
        sc / max(tc, 0.5),  
        1.0 if sc >= tc - 0.5 else 0.0,  
        abs(sc - tc),  
        slab,  
        *src_oh,  
        *tgt_oh,  
    ]
```

```
def _legacy_to_pair(f8):  
    return f8[:4], [f8[4], f8[5], f8[6], 0]
```

```
def build_blocking_index(course_list):  
    index = defaultdict(list)  
    for course in course_list:  
        subject_code = int(course[2])  
        index[subject_code].append(course)  
    return dict(index)
```

```
class TransferMatcherNet(nn.Module):  
    def __init__(self, input_size=FEATURE_SIZE):  
        super().__init__()  
        self.network = nn.Sequential(  
            nn.Linear(input_size, 64),  
            nn.BatchNorm1d(64),  
            nn.ReLU(),  
            nn.Dropout(0.3),  
            nn.Linear(64, 32),  
            nn.BatchNorm1d(32),  
            nn.ReLU(),  
            nn.Dropout(0.2),  
            nn.Linear(32, 16),
```

```

        nn.ReLU(),
        nn.Linear(16, 1),
        nn.Sigmoid(),
    )

    def forward(self, x):
        return self.network(x)

def _compute_norm(X):
    mean = X.mean(dim=0)
    std = X.std(dim=0)
    std[std < 1e-6] = 1.0
    return mean, std

def _normalize(X, mean, std):
    return (X - mean) / std

def get_sample_training_data():
    return [
        ([3, 0, 0, 0], [3, 0, 0, 0], 1),
        ([3, 1, 0, 0], [3, 1, 0, 0], 1),
        ([4, 1, 1, 1], [4, 1, 1, 0], 1),
        ([3, 0, 2, 0], [3, 0, 2, 0], 1),
        ([4, 2, 4, 0], [4, 2, 4, 0], 1),
        ([3, 0, 3, 0], [3, 0, 3, 0], 1),
        ([3, 1, 4, 0], [3, 1, 4, 0], 1),
        ([4, 0, 1, 1], [4, 0, 1, 0], 1),
        ([3, 3, 4, 0], [3, 3, 4, 0], 1),
        ([3, 0, 5, 0], [3, 0, 5, 0], 1),
        ([3, 1, 0, 0], [3, 0, 0, 0], 1),
        ([4, 2, 1, 1], [3, 1, 1, 0], 1),
        ([3, 3, 0, 0], [3, 2, 0, 0], 1),
        ([4, 3, 4, 0], [3, 2, 4, 0], 1),
        ([3, 2, 2, 0], [3, 1, 2, 0], 1),
        ([4, 1, 1, 1], [3, 0, 1, 0], 1),
        ([3, 0, 4, 0], [3, 1, 4, 0], 0),
        ([3, 1, 0, 0], [3, 2, 0, 0], 0),
        ([3, 2, 4, 0], [3, 3, 4, 0], 0),
        ([3, 0, 2, 0], [3, 1, 2, 0], 0),
        ([3, 0, 0, 0], [4, 0, 0, 0], 0),
        ([1, 0, 0, 0], [4, 0, 0, 0], 0),
        ([2, 1, 2, 0], [5, 1, 2, 0], 0),
        ([3, 0, 0, 0], [3, 0, 1, 0], 0),
        ([3, 0, 2, 0], [3, 0, 0, 0], 0),
        ([4, 1, 4, 0], [4, 1, 1, 0], 0),
        ([3, 0, 3, 0], [3, 0, 4, 0], 0),
        ([3, 1, 1, 1], [3, 1, 4, 0], 0),
    ]

```

```

([4, 2, 5, 0], [4, 2, 0, 0], 0),
([3, 1, 0, 0], [3, 1, 3, 0], 0),
([4, 0, 4, 0], [4, 0, 2, 0], 0),
([3, 0, 0, 0], [3, 0, 0, 1], 1),
([4, 2, 1, 0], [4, 2, 1, 1], 1),
([3.5, 0, 0, 0], [3, 0, 0, 0], 1),
([2.5, 0, 0, 0], [3, 0, 0, 0], 1),
]

```

```

def _samples_to_tensors(samples):
    fvs, labels = [], []
    for s in samples:
        if isinstance(s, dict):
            if "source" in s and "target" in s:
                fv = build_feature_vector(s["source"], s["target"])
            elif "features" in s:
                f = s["features"]
                fv = build_feature_vector(*_legacy_to_pair(f)) if
len(f) == 8 else f
            else:
                continue
            fvs.append(fv)
            labels.append(float(s["label"]))
        elif isinstance(s, (list, tuple)) and len(s) == 3:
            fvs.append(build_feature_vector(s[0], s[1]))
            labels.append(float(s[2]))
        elif isinstance(s, (list, tuple)) and len(s) == 2:
            f, lbl = s
            fv = build_feature_vector(*_legacy_to_pair(f)) if len(f)
== 8 else f
            fvs.append(fv)
            labels.append(float(lbl))
    return (
        torch.tensor(fvs, dtype=torch.float32),
        torch.tensor(labels, dtype=torch.float32).unsqueeze(1),
    )

```

```

def train_model(samples=None, epochs=300, learning_rate=0.001,
patience=50):
    if not samples:
        samples = get_sample_training_data()
    X, y = _samples_to_tensors(samples)
    n = len(X)
    feat_mean, feat_std = _compute_norm(X)
    X_norm = _normalize(X, feat_mean, feat_std)
    idx = torch.randperm(n)
    split = max(int(n * 0.8), 1)
    Xt, yt = X_norm[idx[:split]], y[idx[:split]]

```

```

Xv, yv = X_norm[idx[split:]], y[idx[split:]]
has_val = len(Xv) > 0
model = TransferMatcherNet()
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", patience=20, factor=0.5, min_lr=1e-6
)
best_val, pat_cnt, best_state, final_ep = float("inf"), 0, None, 0
for ep in range(epochs):
    model.train()
    optimizer.zero_grad()
    loss = criterion(model(Xt), yt)
    loss.backward()
    optimizer.step()
    if has_val:
        model.eval()
        with torch.no_grad():
            vl = criterion(model(Xv), yv).item()
        scheduler.step(vl)
        if vl < best_val:
            best_val, pat_cnt = vl, 0
            best_state = {k: v.clone() for k, v in
model.state_dict().items()}
        else:
            pat_cnt += 1
            if pat_cnt >= patience:
                break
    final_ep = ep + 1
if best_state:
    model.load_state_dict(best_state)
model.eval()
with torch.no_grad():
    preds = model(X_norm)
    accuracy = ((preds >= 0.5).float() == y).float().mean().item()
    final_loss = criterion(preds, y).item()
torch.save(model.state_dict(), MODEL_PATH)
meta = {
    "isTrained": True,
    "lastTrainedAt":
datetime.datetime.now(datetime.timezone.utc).isoformat(),
    "trainingAccuracy": round(accuracy, 4),
    "epochs": final_ep,
    "finalLoss": round(final_loss, 6),
    "bestValLoss": round(best_val, 6) if has_val else None,
    "numSamples": n,
    "featureSize": FEATURE_SIZE,
    "normalization": {"mean": feat_mean.tolist(), "std":
feat_std.tolist()},
}

```

```

with open(MODEL_META_PATH, "w", encoding="utf-8") as f:
    json.dump(meta, f, indent=2)
global _cached_model, _cached_norm
_cached_model = _cached_norm = None
return {
    "success": True,
    "epochs": final_ep,
    "finalLoss": round(final_loss, 6),
    "accuracy": round(accuracy, 4),
    "message": f"Training complete on {n} samples.",
}

```

```

_cached_model = None
_cached_norm = None

```

```

def _load_model():
    global _cached_model, _cached_norm
    if _cached_model is not None:
        return _cached_model, _cached_norm
    if not os.path.exists(MODEL_PATH):
        raise RuntimeError("Model not trained. Run train_model
first.")
    m = TransferMatcherNet()
    m.load_state_dict(torch.load(MODEL_PATH, weights_only=True))
    m.eval()
    norm = (torch.zeros(FEATURE_SIZE), torch.ones(FEATURE_SIZE))
    if os.path.exists(MODEL_META_PATH):
        with open(MODEL_META_PATH, encoding="utf-8") as f:
            nd = json.load(f).get("normalization", {})
            if "mean" in nd and "std" in nd:
                norm = (
                    torch.tensor(nd["mean"], dtype=torch.float32),
                    torch.tensor(nd["std"], dtype=torch.float32),
                )
    _cached_model, _cached_norm = m, norm
    return m, norm

```

```

def _classify(prob):
    conf = "high" if prob >= 0.75 or prob <= 0.25 else "medium" if
prob >= 0.60 or prob <= 0.40 else "low"
    return {"matchProbability": round(prob, 4), "isMatch": prob >=
0.5, "confidence": conf}

```

```

def predict(features=None, source=None, target=None):
    model, (mean, std) = _load_model()
    if source is not None and target is not None:

```

```

        fv = build_feature_vector(source, target)
    elif features is not None:
        fv = build_feature_vector(*_legacy_to_pair(features)) if
len(features) == 8 else features
    else:
        raise ValueError("Provide (source, target) or features")
    X = _normalize(torch.tensor([fv], dtype=torch.float32), mean, std)
    with torch.no_grad():
        return _classify(model(X).item())

def predict_batch(pairs):
    model, (mean, std) = _load_model()
    fvs = []
    for p in pairs:
        if "source" in p and "target" in p:
            fvs.append(build_feature_vector(p["source"], p["target"]))
        elif "features" in p:
            f = p["features"]
            fvs.append(build_feature_vector(*_legacy_to_pair(f)) if
len(f) == 8 else f)
    if not fvs:
        return []
    X = _normalize(torch.tensor(fvs, dtype=torch.float32), mean, std)
    with torch.no_grad():
        probs = model(X).squeeze(-1).tolist()
    if isinstance(probs, float):
        probs = [probs]
    return [_classify(p) for p in probs]

def predict_batch_blocked(source_course, blocking_index):
    subject_code = int(source_course[2])
    candidates = blocking_index.get(subject_code, [])
    if not candidates:
        return []
    pairs = [{"source": source_course, "target": tgt} for tgt in
candidates if tgt != source_course]
    if not pairs:
        return []
    results = predict_batch(pairs)
    scored = []
    for pair, result in zip(pairs, results):
        scored.append(
            {
                "target": pair["target"],
                "matchProbability": result["matchProbability"],
                "isMatch": result["isMatch"],
                "confidence": result["confidence"],
            }
        )

```

```

    )
    scored.sort(key=lambda x: x["matchProbability"], reverse=True)
    return scored

def find_matches_for_courses(source_courses, target_courses):
    blocking_index = build_blocking_index(target_courses)
    all_matches = {}
    for i, src in enumerate(source_courses):
        all_matches[i] = predict_batch_blocked(src, blocking_index)
    return all_matches

def get_status():
    if os.path.exists(MODEL_META_PATH):
        with open(MODEL_META_PATH, encoding="utf-8") as f:
            meta = json.load(f)
            meta["modelPath"] = MODEL_PATH
            return meta
    return {"isTrained": False, "modelPath": MODEL_PATH,
"featureSize": FEATURE_SIZE}

SUBJECT_MAP = {
    "math": 0,
    "mathematics": 0,
    "science": 1,
    "biology": 1,
    "chemistry": 1,
    "physics": 1,
    "english": 2,
    "writing": 2,
    "composition": 2,
    "history": 3,
    "social": 3,
    "cs": 4,
    "computer": 4,
    "programming": 4,
    "information": 4,
    "technology": 4,
}

def _encode_subject(s):
    if not isinstance(s, str):
        return 5
    s = s.lower().strip()
    for kw, code in SUBJECT_MAP.items():
        if kw in s:
            return code

```

```
return 5
```

```
def _encode_level(val):  
    if val is None:  
        return 0  
    nums = re.findall(r"\d+", str(val))  
    if nums:  
        n = int(nums[0])  
        if n >= 400:  
            return 3  
        if n >= 300:  
            return 2  
        if n >= 200:  
            return 1  
    return 0
```

```
def _clean_text(text):  
    if not isinstance(text, str):  
        return ""  
    text = text.lower()  
    text = re.sub(r"^[a-z0-9\\s]", " ", text)  
    return re.sub(r"\\s+", " ", text).strip()
```

```
NLP_FEATURE_SIZE = 14  
NLP_MODEL_PATH = os.path.join(os.path.dirname(__file__),  
    "model_nlp_weights.pt")  
NLP_MODEL_META_PATH = os.path.join(os.path.dirname(__file__),  
    "model_nlp_meta.json")  
DEFAULT_CSV_DIR = os.path.join(os.path.dirname(__file__), "..",  
    "csv_exports")
```

```
def load_ku_data(csv_dir=DEFAULT_CSV_DIR):  
    import pandas as pd  
    csv_dir = os.path.abspath(csv_dir)  
    paths = {  
        "courses": os.path.join(csv_dir, "courses.csv"),  
        "knowledge_units": os.path.join(csv_dir,  
"knowledge_units.csv"),  
        "course_ku": os.path.join(csv_dir, "course_ku.csv"),  
    }  
    for name, p in paths.items():  
        if not os.path.exists(p):  
            raise FileNotFoundError(f"Missing CSV '{name}': {p}")  
    courses_df = pd.read_csv(paths["courses"])  
    ku_df = pd.read_csv(paths["knowledge_units"])  
    ck_df = pd.read_csv(paths["course_ku"])
```

```

    ku_map = {
        int(r["ku_id"]): {"name": str(r.get("ku_name", "")),
            "description": str(r.get("ku_description", ""))}
        for _, r in ku_df.iterrows()
    }
    course_ku_map = defaultdict(set)
    for _, r in ck_df.iterrows():
        course_ku_map[int(r["course_id"])] += set(int(r["ku_id"]))
    return courses_df, ku_map, course_ku_map

def build_ku_text_profiles(courses_df, ku_map, course_ku_map):
    profiles = {}
    for _, row in courses_df.iterrows():
        cid = int(row["course_id"])
        course_name = _clean_text(str(row.get("course_name", "")))
        ku_ids = course_ku_map.get(cid, set())
        ku_texts = []
        for kid in sorted(ku_ids):
            ku = ku_map.get(kid, {})
            ku_texts.extend([_clean_text(ku.get("name", "")),
                _clean_text(ku.get("description", ""))])
        profiles[cid] = {"name": course_name, "ku_profile": " ".join(
            t for t in ku_texts if t) or "unknown", "ku_ids": ku_ids}
    return profiles

def build_tfidf_vectors(profiles):
    from sklearn.feature_extraction.text import TfidfVectorizer
    course_ids = sorted(profiles.keys())
    ku_texts = [profiles[c]["ku_profile"] for c in course_ids]
    name_texts = [profiles[c]["name"] or "unknown" for c in
        course_ids]
    ku_tfidf = TfidfVectorizer(min_df=1, ngram_range=(1, 2))
    name_tfidf = TfidfVectorizer(min_df=1, ngram_range=(1, 2))
    ku_matrix = ku_tfidf.fit_transform(ku_texts)
    name_matrix = name_tfidf.fit_transform(name_texts)
    return ku_tfidf, name_tfidf, ku_matrix, name_matrix, course_ids

def compute_nlp_features(src_cid, tgt_cid, profiles, ku_matrix,
    name_matrix, course_ids, courses_df):
    from sklearn.metrics.pairwise import cosine_similarity
    cid_idx = {c: i for i, c in enumerate(course_ids)}

    def _get_row(cid):
        rows = courses_df[courses_df["course_id"] == cid]
        return rows.iloc[0] if not rows.empty else None

    def _credits(row):

```

```

    try:
        return float(row["credits"]) if row is not None else 3.0
    except Exception:
        return 3.0

def _level(row):
    if row is None:
        return 0
    nums = re.findall(r"\d+", str(row.get("course_code", "")))
    if nums:
        n = int(nums[0])
        if n >= 400:
            return 3
        if n >= 300:
            return 2
        if n >= 200:
            return 1
    return 0

def _subj(row):
    if row is None:
        return 4
    return _encode_subject(str(row.get("course_name", "")) + " " +
str(row.get("course_code", "")))

sr, tr = _get_row(src_cid), _get_row(tgt_cid)
src_credits, tgt_credits = _credits(sr), _credits(tr)
src_kus = profiles.get(src_cid, {}).get("ku_ids", set())
tgt_kus = profiles.get(tgt_cid, {}).get("ku_ids", set())
union = src_kus | tgt_kus
inter = src_kus & tgt_kus
jaccard = len(inter) / len(union) if union else 0.0
si, ti = cid_idx.get(src_cid), cid_idx.get(tgt_cid)
if si is not None and ti is not None:
    tfidf_cos = float(cosine_similarity(ku_matrix[si],
ku_matrix[ti])[0, 0])
    name_cos = float(cosine_similarity(name_matrix[si],
name_matrix[ti])[0, 0])
else:
    tfidf_cos = name_cos = 0.0
return [
    src_credits,
    float(_level(sr)),
    float(_subj(sr)),
    0.0,
    tgt_credits,
    float(_level(tr)),
    float(_subj(tr)),
    abs(src_credits - tgt_credits),
    jaccard,

```

```

        float(len(src_kus)),
        float(len(tgt_kus)),
        abs(float(len(src_kus)) - float(len(tgt_kus))),
        tfidf_cos,
        name_cos,
    ]

```

```

class NLPTransferMatcherNet(nn.Module):
    def __init__(self, input_size=NLP_FEATURE_SIZE):
        super().__init__()
        self.features = nn.Sequential(
            nn.Linear(input_size, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, 16),
            nn.ReLU(),
        )
        self.head = nn.Linear(16, 1)

    def logits(self, x):
        return self.head(self.features(x))

    def forward(self, x):
        return torch.sigmoid(self.logits(x))

_cached_nlp_model = None
_cached_nlp_norm = None

def build_nlp_training_samples(csv_dir=DEFAULT_CSV_DIR):
    courses_df, ku_map, course_ku_map = load_ku_data(csv_dir)
    profiles = build_ku_text_profiles(courses_df, ku_map,
    course_ku_map)
    _, _, ku_matrix, name_matrix, course_ids =
    build_tfidf_vectors(profiles)
    samples = []
    for src_cid in course_ids:
        for tgt_cid in course_ids:
            if src_cid == tgt_cid:
                continue
            fv = compute_nlp_features(src_cid, tgt_cid, profiles,
    ku_matrix, name_matrix, course_ids, courses_df)
            level_ok = fv[1] >= fv[5]
            credits_ok = fv[0] >= fv[4] - 0.5
            has_kus = fv[9] > 0 or fv[10] > 0

```

```

        label = 1.0 if (has_kus and fv[8] >= 0.1 and level_ok and
credits_ok) else 0.0
        samples.append({"features": fv, "label": label})
    return samples

def train_nlp_model(csv_dir=DEFAULT_CSV_DIR, epochs=300,
learning_rate=0.001, patience=50):
    samples = build_nlp_training_samples(csv_dir)
    if not samples:
        return {"success": False, "message": "No training samples
generated."}
    X = torch.tensor([s["features"] for s in samples],
dtype=torch.float32)
    y = torch.tensor([s["label"] for s in samples],
dtype=torch.float32).unsqueeze(1)
    n = len(X)
    feat_mean, feat_std = _compute_norm(X)
    X_norm = _normalize(X, feat_mean, feat_std)
    idx = torch.randperm(n)
    split = max(int(n * 0.8), 1)
    Xt, yt = X_norm[idx[:split]], y[idx[:split]]
    Xv, yv = X_norm[idx[split:]], y[idx[split:]]
    has_val = len(Xv) > 0
    model = NLPTransferMatcherNet()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode="min", patience=20, factor=0.5, min_lr=1e-6)
    n_pos = y.sum().item()
    n_neg = n - n_pos
    pos_weight = torch.tensor([n_neg / max(n_pos, 1)],
dtype=torch.float32)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    best_val, pat_cnt, best_state, final_ep = float("inf"), 0, None, 0
    for ep in range(epochs):
        model.train()
        optimizer.zero_grad()
        loss = criterion(model.logits(Xt), yt)
        loss.backward()
        optimizer.step()
        if has_val:
            model.eval()
            with torch.no_grad():
                vl = criterion(model.logits(Xv), yv).item()
            scheduler.step(vl)
            if vl < best_val:
                best_val, pat_cnt = vl, 0
                best_state = {k: v.clone() for k, v in
model.state_dict().items()}
            else:

```

```

        pat_cnt += 1
        if pat_cnt >= patience:
            break
    final_ep = ep + 1
    if best_state:
        model.load_state_dict(best_state)
    model.eval()
    bce = nn.BCELoss()
    with torch.no_grad():
        preds = model(X_norm)
        accuracy = ((preds >= 0.5).float() == y).float().mean().item()
        final_loss = bce(preds, y).item()
    torch.save(model.state_dict(), NLP_MODEL_PATH)
    meta = {
        "isTrained": True,
        "lastTrainedAt":
datetime.datetime.now(datetime.timezone.utc).isoformat(),
        "trainingAccuracy": round(accuracy, 4),
        "epochs": final_ep,
        "finalLoss": round(final_loss, 6),
        "bestValLoss": round(best_val, 6) if has_val else None,
        "numSamples": n,
        "featureSize": NLP_FEATURE_SIZE,
        "normalization": {"mean": feat_mean.tolist(), "std":
feat_std.tolist()},
    }
    with open(NLP_MODEL_META_PATH, "w", encoding="utf-8") as f:
        json.dump(meta, f, indent=2)
    global _cached_nlp_model, _cached_nlp_norm
    _cached_nlp_model = _cached_nlp_norm = None
    return {"success": True, "epochs": final_ep, "finalLoss":
round(final_loss, 6), "accuracy": round(accuracy, 4), "numSamples": n}

def _load_nlp_model():
    global _cached_nlp_model, _cached_nlp_norm
    if _cached_nlp_model is not None:
        return _cached_nlp_model, _cached_nlp_norm
    if not os.path.exists(NLP_MODEL_PATH):
        raise RuntimeError("NLP model not trained. Run train_nlp_model
first.")
    m = NLPTransferMatcherNet()
    m.load_state_dict(torch.load(NLP_MODEL_PATH, weights_only=True))
    m.eval()
    norm = (torch.zeros(NLP_FEATURE_SIZE),
torch.ones(NLP_FEATURE_SIZE))
    if os.path.exists(NLP_MODEL_META_PATH):
        with open(NLP_MODEL_META_PATH, encoding="utf-8") as f:
            nd = json.load(f).get("normalization", {})
            if "mean" in nd and "std" in nd:

```

```

        norm = (torch.tensor(nd["mean"], dtype=torch.float32),
torch.tensor(nd["std"], dtype=torch.float32))
        _cached_nlp_model, _cached_nlp_norm = m, norm
        return m, norm

def predict_nlp(src_course_id, tgt_course_id,
csv_dir=DEFAULT_CSV_DIR):
    model, (mean, std) = _load_nlp_model()
    courses_df, ku_map, course_ku_map = load_ku_data(csv_dir)
    profiles = build_ku_text_profiles(courses_df, ku_map,
course_ku_map)
    _, _, ku_matrix, name_matrix, course_ids =
build_tfidf_vectors(profiles)
    fv = compute_nlp_features(src_course_id, tgt_course_id, profiles,
ku_matrix, name_matrix, course_ids, courses_df)
    X = _normalize(torch.tensor([fv], dtype=torch.float32), mean, std)
    with torch.no_grad():
        prob = model(X).item()
        result = _classify(prob)
        result["sourceId"] = src_course_id
        result["targetId"] = tgt_course_id
        src_kus = profiles.get(src_course_id, {}).get("ku_ids", set())
        tgt_kus = profiles.get(tgt_course_id, {}).get("ku_ids", set())
        union = src_kus | tgt_kus
        inter = src_kus & tgt_kus
        result["kuOverlapRatio"] = round(len(inter) / len(union), 4) if
union else 0.0
        result["tfidfCosineSim"] = round(fv[12], 4)
        result["nameCosineSim"] = round(fv[13], 4)
        return result

def analyze_kus(src_course_id, tgt_course_id,
csv_dir=DEFAULT_CSV_DIR):
    from sklearn.metrics.pairwise import cosine_similarity
    courses_df, ku_map, course_ku_map = load_ku_data(csv_dir)
    profiles = build_ku_text_profiles(courses_df, ku_map,
course_ku_map)
    _, _, ku_matrix, name_matrix, course_ids =
build_tfidf_vectors(profiles)
    cid_idx = {c: i for i, c in enumerate(course_ids)}

    def _name(cid):
        rows = courses_df[courses_df["course_id"] == cid]
        return rows.iloc[0]["course_name"] if not rows.empty else
f"Course {cid}"

    src_kus = profiles.get(src_course_id, {}).get("ku_ids", set())
    tgt_kus = profiles.get(tgt_course_id, {}).get("ku_ids", set())

```

```

shared = src_kus & tgt_kus
src_only = src_kus - tgt_kus
tgt_only = tgt_kus - src_kus
union = src_kus | tgt_kus
jaccard = len(shared) / len(union) if union else 0.0
si, ti = cid_idx.get(src_course_id), cid_idx.get(tgt_course_id)
tfidf_cos = float(cosine_similarity(ku_matrix[si], ku_matrix[ti])
[0, 0]) if si is not None and ti is not None else 0.0
name_cos = float(cosine_similarity(name_matrix[si],
name_matrix[ti])[0, 0]) if si is not None and ti is not None else 0.0

def _ku_list(ids):
    return [{"id": k, "name": ku_map.get(k, {}).get("name", "")}
for k in sorted(ids)]

return {
    "sourceCourse": {"id": src_course_id, "name":
_name(src_course_id), "kuCount": len(src_kus), "kus":
_ku_list(src_kus)},
    "targetCourse": {"id": tgt_course_id, "name":
_name(tgt_course_id), "kuCount": len(tgt_kus), "kus":
_ku_list(tgt_kus)},
    "sharedKUs": _ku_list(shared),
    "srcOnlyKUs": _ku_list(src_only),
    "tgtOnlyKUs": _ku_list(tgt_only),
    "kuOverlapRatio": round(jaccard, 4),
    "tfidfCosineSim": round(tfidf_cos, 4),
    "nameCosineSim": round(name_cos, 4),
    "suggestedLabel": 1 if jaccard >= 0.2 else 0,
}

def main():
    parser = argparse.ArgumentParser(prog="transfer_matcher_full.py")
    sub = parser.add_subparsers(dest="command")
    p_t = sub.add_parser("train")
    p_t.add_argument("--epochs", type=int, default=300)
    p_t.add_argument("--lr", type=float, default=0.001)
    p_t.add_argument("--patience", type=int, default=50)
    p_t.add_argument("--samples", type=str, default=None)
    p_p = sub.add_parser("predict")
    p_p.add_argument("payload", nargs="?", default="{}")
    sub.add_parser("status")
    p_tn = sub.add_parser("train_nlp")
    p_tn.add_argument("--csv-dir", default=DEFAULT_CSV_DIR)
    p_tn.add_argument("--epochs", type=int, default=300)
    p_tn.add_argument("--lr", type=float, default=0.001)
    p_tn.add_argument("--patience", type=int, default=50)
    p_pn = sub.add_parser("predict_nlp")
    p_pn.add_argument("--source", type=int, required=True)

```

```

p_pn.add_argument("--target", type=int, required=True)
p_pn.add_argument("--csv-dir", default=DEFAULT_CSV_DIR)
p_ak = sub.add_parser("analyze_kus")
p_ak.add_argument("--source", type=int, required=True)
p_ak.add_argument("--target", type=int, required=True)
p_ak.add_argument("--csv-dir", default=DEFAULT_CSV_DIR)
args = parser.parse_args()
if args.command == "train":
    samples = None
    if args.samples:
        with open(args.samples, encoding="utf-8") as f:
            samples = json.load(f)
        print(json.dumps(train_model(samples=samples,
epochs=args.epochs, learning_rate=args.lr, patience=args.patience),
indent=2))
    elif args.command == "predict":
        payload = json.loads(args.payload)
        src, tgt, feats = payload.get("source"),
payload.get("target"), payload.get("features")
        if src and tgt:
            result = predict(source=src, target=tgt)
        elif feats:
            result = predict(features=feats)
        else:
            raise SystemExit("Provide source/target or features.")
        print(json.dumps(result, indent=2))
    elif args.command == "status":
        print(json.dumps(get_status(), indent=2))
    elif args.command == "train_nlp":
        print(json.dumps(train_nlp_model(csv_dir=args.csv_dir,
epochs=args.epochs, learning_rate=args.lr, patience=args.patience),
indent=2))
    elif args.command == "predict_nlp":
        print(json.dumps(predict_nlp(args.source, args.target,
csv_dir=args.csv_dir), indent=2))
    elif args.command == "analyze_kus":
        print(json.dumps(analyze_kus(args.source, args.target,
csv_dir=args.csv_dir), indent=2))
    else:
        parser.print_help()

```

```

if __name__ == "__main__":
    main()

```

```

=====
=====
FILE: readme.txt
=====
=====

```

```
=====
=====
TRANSFER CREDIT MATCH – Data Science Branch
Proof of Concept Documentation
=====
=====
```

```
BRANCH: Data_Science
LAST UPDATED: April 2026
TEAM ROLE: Data Science / Database Pipeline
```

```
-----
OVERVIEW
-----
```

This branch contains the Data Science layer of the Transfer Credit Match system.
The goal of this component is to support intelligent course-to-course matching across institutions using structured academic data, knowledge unit (KU) mappings, and a PDF transcript parser.

The system helps students who transfer between Chicago-area colleges identify which courses they have already completed and which ones may satisfy requirements at their destination institution.

```
-----
FOLDER STRUCTURE
-----
```

```
Data_Science/
├── readme.txt                ← You are here
├── TranscriptPDF_reader/
│   └── pdf_reader.py        ← PDF transcript parser script
├── csv_exports/
│   ├── institutions.csv     ← Participating institutions
│   ├── programs.csv        ← Academic programs per institution
│   ├── courses.csv         ← Course catalog across institutions
│   ├── knowledge_units.csv ← KU taxonomy (CAE-aligned)
│   ├── course_ku.csv       ← Course ↔ KU mapping (many-to-many)
│   ├── students.csv        ← Student enrollment records
│   └── transfer_requests.csv ← Student-submitted transfer requests
```

— course_match.csv	← Matched course pairs (with status)
— match_history.csv	← Historical match audit log
— admins.csv	← Admin user records
— directors.csv	← Program director records
— users.csv	← All system users

INSTITUTIONS (Proof of Concept)

The system currently seeds four real Chicago-area institutions:

ID	Institution	Location
1	Roosevelt University	Chicago, IL
2	National Louis University	Chicago, IL
3	University of Chicago	Chicago, IL
4	Harold Washington College	Chicago, IL

ACADEMIC PROGRAMS

Each institution offers one or more programs that students enroll in:

Program ID	Institution	Program Name
1	Roosevelt University	Computer Science
2	Roosevelt University	Cybersecurity
3	Harold Washington College	Information Technology
4	Harold Washington College	Software Development
5	National Louis University	Computer Forensics
6	National Louis University	Computer Science
7	University of Chicago	Cloud Computing
8	University of Chicago	Information Technology
9	Roosevelt University	Cyber Security & Info

Assurance

COURSE CATALOG (Sample – 25 Courses Seeded)

Courses are tied to institutions and programs. Below is a sample:

Course ID	Code	Course Name	Credits
1	CS101	Data Structures and Algorithms	4
2	CYB201	Network Security	3
3	IT101	IT Fundamentals	3
4	SD102	Web Development	3
5	CSIA 150	Computer Science I	4
6	CSIA 217	Intro to Prob. & Stats	3
7	CSIA 236	Python Script Programming	3
8	CSIA 246	Data Communications	3
9	CSIA 250	Computer Science II	4
10	CSIA 255	Open Source Communities	3
11	CSIA 261	Computer Organization	3
12	CSIA 301	Computer Networking	3
13	CSIA 317	Operating Systems	3
14	CSIA 318	UNIX and System Administration	3
15	CSIA 319	Cyber Ops	3
16	CSIA 327	Software Engineering	3
17	CSIA 333	Database Systems	3
18	CSIA 335	Ethical Hack & Countermeasures	3
19	CSIA 336	Practical Computing with Data/Python	3
20	CSIA 352	Network Design	3
...	...	(+ 5 more courses seeded)	
...			

KNOWLEDGE UNITS (KU TAXONOMY – CAE-Aligned)

Knowledge Units represent standardized academic competency areas. Each course is mapped to one or more KUs to enable semantic matching across institutions.

KU ID	Knowledge Unit Name	Description
1	Programming Concepts and syntax	Fundamental logic
2	Cybersecurity Basics concepts	Intro security
3	Networking Fundamentals	Basic networking

principles		
4	Database Management	SQL and NoSQL
principles		
5	Cybersecurity Foundations	Core
cybersecurity concepts		
6	Cybersecurity Principles	Security design
fundamentals		
7	IT Systems Components	IT system roles/
operations		
8	Basic Scripting and Programming	Script/program
creation basics		
9	Basic Networking	Network build/
operate/analyze		
10	Network Defense	Network threat
protection		
11	Basic Cryptography	Cryptography
usage		
12	Operating Systems Concepts	OS roles and
services		
13	Advanced Network Technology & Protocols	Advanced
networking security		
14	Cloud Computing	Cloud models and
security		
15	Cybersecurity Ethics	Ethics in cyber
contexts		
16	Database Management Systems	DBMS utilization
skills		
17	Databases	DB usage,
management, issues		
18	Digital Communications	Modern digital
comm protocols		
19	Intrusion Detection/Prevention Systems	Vulnerability
detection/mitigation		
20	Low Level Programming	Secure low-level
programming		
21	Network Security Administration	Enterprise
security infra		
22	Network Technology and Protocols	Common protocols/
interactions		
23	Operating Systems Administration	OS system admin
operations		
24	Operating Systems Theory	OS concepts/
components/interfaces		
25	Vulnerability Analysis	System
vulnerability coverage		
26	Web Application Security	Web app tools and
practices		

Total KUs seeded: 26

COURSE ↔ KNOWLEDGE UNIT MAPPING (Many-to-Many)

Courses are mapped to multiple KUs to enable cross-institution matching.

Example mappings:

Course ID	Course Name	Mapped KU IDs
1	Data Structures & Algorithms Concepts)	KU 1 (Programming
2	Network Security Basics)	KU 2 (Cybersecurity
3	IT Fundamentals Fundamentals)	KU 3 (Networking
4	Web Development Management)	KU 4 (Database
23	Intro to Computer Security	KU 5, 6, 11, 15, 19, 25,
26		
13	Operating Systems	KU 7, 12, 14, 24
12	Computer Networking	KU 9, 13, 18, 22
17	Database Systems	KU 16, 17
24	Internet Security	KU 10, 21, 24, 26

Total course-KU links seeded: 32

→ This mapping is the core driver of the matching algorithm. Two courses from different institutions are considered equivalent when they share a sufficient overlap of Knowledge Units.

PROOF OF CONCEPT – TRANSFER REQUEST EXAMPLE

The following is a real end-to-end example seeded in the system:

STUDENT

Student ID : 1

Enrolled at: Roosevelt University (Institution 1)

Program : Computer Science (Program 1)

TRANSFER REQUEST

Request ID : 1
From : Roosevelt University (Institution 1)
To : Harold Washington College (Institution 4)
Course From : CS101 – Data Structures and Algorithms (Course 1, 4 credits)
Course To : SD102 – Web Development (Course 4, 3 credits)
Status : Pending
Date Submitted : 2026-02-16

COURSE MATCH RECORD

Match ID : 1
Institution From: Roosevelt University → Harold Washington College
Course From : CS101 – Data Structures and Algorithms
Course To : SD102 – Web Development
Match Status : Pending

→ When the system processes this request, it compares the KU profiles of CS101 and SD102. If enough KU overlap is found, the match is approved.

A director at Harold Washington College reviews and finalizes the decision.

PDF TRANSCRIPT PARSER

File: TranscriptPDF_reader/pdf_reader.py

PURPOSE:

Parses student academic transcripts in PDF format (Banner-based and web-saved PDFs) to extract:

- Course codes
- Course names
- Grades
- Credit hours

The extracted data is used to auto-populate transfer request submissions, reducing manual entry errors and speeding up the matching workflow.

USAGE:

python pdf_reader.py --pdf <path_to_transcript.pdf>

SUPPORTED FORMATS:

- Banner-generated PDF transcripts
- Web-saved/printed PDF transcripts (with layout noise handling)

LIBRARIES USED:

- pdfplumber - for table-based extraction from structured PDFs
- re - for regex-based fallback parsing
- pandas - for structuring and exporting extracted data

DATA PIPELINE SUMMARY

```
[Student PDF Transcript]
      ↓
[pdf_reader.py] - parse courses from transcript
      ↓
[Transfer Request submitted to DB]
      ↓
[KU Mapping engine] - compare KU profiles of source vs target course
      ↓
[course_match record created] - status: Pending
      ↓
[Director reviews & approves/denies]
      ↓
[match_history updated] - audit trail preserved
```

CONTRIBUTORS (Data Science Branch)

- Data modeling and CSV seed data
- PDF transcript parsing pipeline
- Knowledge Unit taxonomy design
- Course-to-KU mapping for CAE-aligned programs

END OF DOCUMENT

FILE: requirements.txt

```
=====
```

```
pdfplumber  
pandas  
pytest
```

```
=====
```

```
=====
```

```
FILE: scripts/build_sqlite.py
```

```
=====
```

```
=====
```

```
from __future__ import annotations
```

```
import argparse  
import sqlite3  
from pathlib import Path
```

```
import pandas as pd
```

```
TABLE_FILES = {  
    "institutions": "institutions.csv",  
    "programs": "programs.csv",  
    "courses": "courses.csv",  
    "knowledge_units": "knowledge_units.csv",  
    "course_ku": "course_ku.csv",  
    "students": "students.csv",  
    "transfer_requests": "transfer_requests.csv",  
    "course_match": "course_match.csv",  
    "match_history": "match_history.csv",  
    "admins": "admins.csv",  
    "directors": "directors.csv",  
    "users": "users.csv",  
}
```

```
def parse_args() -> argparse.Namespace:  
    parser = argparse.ArgumentParser(description="Build SQLite DB from  
Data_Science CSV exports.")  
    parser.add_argument(  
        "--csv-root", default="csv_exports", help="Directory  
containing input CSV files"  
    )  
    parser.add_argument(  
        "--db-path", default="transfer_credit_match.db", help="Output  
SQLite database path"  
    )  
    return parser.parse_args()
```

```

def main() -> None:
    args = parse_args()
    csv_root = Path(args.csv_root).resolve()
    db_path = Path(args.db_path).resolve()

    with sqlite3.connect(db_path) as conn:
        for table_name, csv_name in TABLE_FILES.items():
            csv_path = csv_root / csv_name
            frame = pd.read_csv(csv_path)
            frame.to_sql(table_name, conn, if_exists="replace",
index=False)
        print(f"Loaded {table_name:<18} rows={len(frame):>4} from
{csv_name}")

    print(f"\nSQLite database built at: {db_path}")

```

```

if __name__ == "__main__":
    main()

```

```

=====
=====
FILE: scripts/run_transfer_match.py
=====
=====

```

```

from __future__ import annotations

import argparse
from pathlib import Path

from engine import load_data_bundle, match_transcript_to_institution
from engine.config import load_matching_rules
from TranscriptPDF_reader.pdf_reader import extract_transcript_data

```

```

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run transcript-to-course transfer matching
against seeded CSV data."
    )
    parser.add_argument("--pdf", required=True, help="Path to
transcript PDF")
    parser.add_argument(
        "--csv-root",
        default="csv_exports",
        help="Directory containing seeded CSV exports (default:
csv_exports)",
    )

```

```

parser.add_argument(
    "--target-institution-id",
    required=True,
    type=int,
    help="Destination institution_id from institutions.csv",
)
parser.add_argument(
    "--top-k",
    type=int,
    default=3,
    help="Top target matches to keep for each source course",
)
parser.add_argument(
    "--threshold",
    type=float,
    default=0.55,
    help="Score threshold for auto-equivalent decision",
)
parser.add_argument(
    "--output",
    default="match_output.csv",
    help="Output CSV path for match results",
)
parser.add_argument(
    "--rules",
    default="config/matching_rules.json",
    help="Matching rules JSON path (default: config/
matching_rules.json)",
)
return parser.parse_args()

```

```

def main() -> None:
    args = parse_args()
    pdf_path = Path(args.pdf).resolve()
    csv_root = Path(args.csv_root).resolve()
    output_path = Path(args.output).resolve()
    rules = load_matching_rules(args.rules)

    transcript_data = extract_transcript_data(str(pdf_path))
    data_bundle = load_data_bundle(csv_root)
    result_df = match_transcript_to_institution(
        transcript_data=transcript_data,
        target_institution_id=args.target_institution_id,
        data_bundle=data_bundle,
        top_k=args.top_k,
        rules=rules,
        approval_threshold=args.threshold,
    )

```

```

        if result_df.empty:
            print("No matches produced. Check transcript course codes
against courses.csv.")
            return

```

```

        result_df.to_csv(output_path, index=False)
        print(f"Saved {len(result_df)} match rows to: {output_path}")
        print(result_df.head(10).to_string(index=False))

```

```

if __name__ == "__main__":
    main()

```

```

=====
=====
FILE: scripts/validate_data.py
=====
=====

```

```

from __future__ import annotations

```

```

import argparse
from pathlib import Path

```

```

import pandas as pd

```

```

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Validate
Data_Science CSV integrity.")
    parser.add_argument("--csv-root", default="csv_exports", help="CSV
directory")
    return parser.parse_args()

```

```

def _load(root: Path, name: str) -> pd.DataFrame:
    return pd.read_csv(root / name)

```

```

def main() -> None:
    args = parse_args()
    root = Path(args.csv_root).resolve()

    institutions = _load(root, "institutions.csv")
    programs = _load(root, "programs.csv")
    courses = _load(root, "courses.csv")
    ku = _load(root, "knowledge_units.csv")
    course_ku = _load(root, "course_ku.csv")
    students = _load(root, "students.csv")
    requests = _load(root, "transfer_requests.csv")

```

```

matches = _load(root, "course_match.csv")

errors: list[str] = []

if institutions["institution_id"].duplicated().any():
    errors.append("Duplicate institution_id values in
institutions.csv")
if courses["course_id"].duplicated().any():
    errors.append("Duplicate course_id values in courses.csv")
if ku["ku_id"].duplicated().any():
    errors.append("Duplicate ku_id values in knowledge_units.csv")

invalid_program_institutions = set(programs["institution_id"]) -
set(institutions["institution_id"])
if invalid_program_institutions:
    errors.append(f"programs.csv has unknown institution_id
values: {sorted(invalid_program_institutions)}")

invalid_course_programs = set(courses["program_id"]) -
set(programs["program_id"])
if invalid_course_programs:
    errors.append(f"courses.csv has unknown program_id values:
{sorted(invalid_course_programs)}")

invalid_course_ku_course = set(course_ku["course_id"]) -
set(courses["course_id"])
if invalid_course_ku_course:
    errors.append(f"course_ku.csv has unknown course_id values:
{sorted(invalid_course_ku_course)}")

invalid_course_ku_ku = set(course_ku["ku_id"]) - set(ku["ku_id"])
if invalid_course_ku_ku:
    errors.append(f"course_ku.csv has unknown ku_id values:
{sorted(invalid_course_ku_ku)}")

invalid_student_institutions = set(students["institution_id"]) -
set(institutions["institution_id"])
if invalid_student_institutions:
    errors.append(f"students.csv has unknown institution_id
values: {sorted(invalid_student_institutions)}")

invalid_request_courses = (set(requests["course_from"]) |
set(requests["course_to"])) - set(courses["course_id"])
if invalid_request_courses:
    errors.append(f"transfer_requests.csv has unknown course ids:
{sorted(invalid_request_courses)}")

invalid_match_courses = (set(matches["course_from"]) |
set(matches["course_to"])) - set(courses["course_id"])
if invalid_match_courses:

```

```

        errors.append(f"course_match.csv has unknown course ids:
{sorted(invalid_match_courses)}")

    if errors:
        print("VALIDATION FAILED")
        for err in errors:
            print(f"- {err}")
        raise SystemExit(1)

    print("VALIDATION PASSED")
    print(f"institutions={len(institutions)} programs={len(programs)}
courses={len(courses)}")
    print(f"knowledge_units={len(ku)}
course_ku_links={len(course_ku)}")
    print(f"students={len(students)} transfer_requests={len(requests)}
course_match={len(matches)}")

if __name__ == "__main__":
    main()

```

```

=====
=====
FILE: scripts/workflow_cli.py
=====
=====

```

```

from __future__ import annotations

import argparse
from pathlib import Path

import pandas as pd

from engine import apply_match_decision,
create_transfer_request_and_match

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Transfer workflow
CLI (submit/review/report).")
    sub = parser.add_subparsers(dest="cmd", required=True)

    submit = sub.add_parser("submit", help="Submit transfer request
and create pending match")
    submit.add_argument("--csv-root", default="csv_exports")
    submit.add_argument("--student-id", required=True, type=int)
    submit.add_argument("--institution-from", required=True, type=int)
    submit.add_argument("--institution-to", required=True, type=int)
    submit.add_argument("--course-from", required=True, type=int)

```

```

        submit.add_argument("--course-to", required=True, type=int)

        review = sub.add_parser("review", help="Apply Approved/Denied
decision to a match")
        review.add_argument("--csv-root", default="csv_exports")
        review.add_argument("--match-id", required=True, type=int)
        review.add_argument("--changed-by", required=True, type=int)
        review.add_argument("--decision", required=True,
choices=["Approved", "Denied"])

        report = sub.add_parser("report", help="Export student match
report")
        report.add_argument("--csv-root", default="csv_exports")
        report.add_argument("--student-id", required=True, type=int)
        report.add_argument("--output",
default="student_transfer_report.csv")

    return parser

def submit_cmd(args: argparse.Namespace) -> None:
    result = create_transfer_request_and_match(
        csv_root=args.csv_root,
        student_id=args.student_id,
        institution_from=args.institution_from,
        institution_to=args.institution_to,
        course_from=args.course_from,
        course_to=args.course_to,
    )
    print(
        f"Submitted request_id={result.request_id}
match_id={result.match_id} "
        f"course_to={result.course_to} status={result.status}"
    )

def review_cmd(args: argparse.Namespace) -> None:
    apply_match_decision(
        csv_root=args.csv_root,
        match_id=args.match_id,
        changed_by=args.changed_by,
        decision=args.decision,
    )
    print(f"Updated match_id={args.match_id} to {args.decision} and
wrote match_history log.")

def report_cmd(args: argparse.Namespace) -> None:
    root = Path(args.csv_root).resolve()
    requests = pd.read_csv(root / "transfer_requests.csv")

```

```

matches = pd.read_csv(root / "course_match.csv")
courses = pd.read_csv(root / "courses.csv")

selected = requests[requests["student_id"] ==
int(args.student_id)].copy()
if selected.empty:
    print(f"No transfer requests found for
student_id={args.student_id}")
    return

merged = selected.merge(
    matches,
    left_on=["course_from", "course_to", "institution_from",
"institution_to"],
    right_on=["course_from", "course_to", "institution_from",
"institution_to"],
    how="left",
    suffixes=("_request", "_match"),
)

merged = merged.merge(
    courses[["course_id", "course_code", "course_name"]],
    left_on="course_from",
    right_on="course_id",
    how="left",
).rename(columns={"course_code": "course_from_code",
"course_name": "course_from_name"})
merged = merged.merge(
    courses[["course_id", "course_code", "course_name"]],
    left_on="course_to",
    right_on="course_id",
    how="left",
    suffixes=("", "_to"),
).rename(columns={"course_code": "course_to_code", "course_name":
"course_to_name"})

cols = [
    "request_id",
    "student_id",
    "institution_from",
    "institution_to",
    "course_from_code",
    "course_from_name",
    "course_to_code",
    "course_to_name",
    "status",
    "match_status",
    "request_date",
    "created_at",
]

```

```

        output = Path(args.output).resolve()
        merged[cols].to_csv(output, index=False)
        print(f"Saved report rows={len(merged)} to {output}")

def main() -> None:
    args = build_parser().parse_args()
    if args.cmd == "submit":
        submit_cmd(args)
    elif args.cmd == "review":
        review_cmd(args)
    elif args.cmd == "report":
        report_cmd(args)

if __name__ == "__main__":
    main()

=====
=====
FILE: start_website.sh
=====
=====

#!/usr/bin/env bash
# Start Transfer Credit Match website: FastAPI backend + Vite React
frontend.
# Usage: from Data_Science folder run:  bash start_website.sh

set -euo pipefail
ROOT="$(cd "$(dirname "$0")" && pwd)"
cd "$ROOT"

echo "== Backend (port 8000) =="
cd "$ROOT/backend"
if [[ ! -d .venv ]]; then
    python3 -m venv .venv
fi
# shellcheck disable=SC1091
source .venv/bin/activate
pip install -q -r requirements.txt
if [[ ! -f .env ]]; then
    cp .env.example .env
fi

cleanup() {
    if [[ -n "${BACK_PID:-}" ]] && kill -0 "$BACK_PID" 2>/dev/null; then
        kill "$BACK_PID" 2>/dev/null || true
    fi
}

```

```
trap cleanup EXIT INT TERM
```

```
python run_api.py &  
BACK_PID=$!
```

```
echo "== Frontend (port 5173) - open http://localhost:5173 =="  
cd "$R00T/frontend"  
if [[ ! -f .env ]]; then  
    cp .env.example .env  
fi  
if [[ ! -d node_modules ]]; then  
    npm install  
fi  
npm run dev
```

```
=====
```

```
=====
```

```
FILE: tests/conftest.py
```

```
=====
```

```
=====
```

```
from __future__ import annotations
```

```
import sys  
from pathlib import Path
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]  
if str(PROJECT_ROOT) not in sys.path:  
    sys.path.insert(0, str(PROJECT_ROOT))
```

```
=====
```

```
=====
```

```
FILE: tests/test_matcher.py
```

```
=====
```

```
=====
```

```
from engine.matcher import compute_course_match
```

```
def test_matcher_equivalent_for_same_ku_and_title():  
    source = {"course_name": "Operating Systems", "course_code": "CSIA  
317", "credits": 3}  
    target = {"course_name": "Operating Systems", "course_code": "CST  
317", "credits": 3}  
  
    result = compute_course_match(  
        source_course=source,  
        target_course=target,  
        source_ku_ids={7, 12, 24},
```

```

        target_ku_ids={7, 12, 24},
        source_syllabus="scheduling memory file systems",
        target_syllabus="memory scheduling file systems",
    )
    assert result.decision == "Equivalent"
    assert result.ku_overlap_count == 3
    assert result.score >= 0.55

def test_matcher_not_equivalent_when_no_ku_overlap():
    source = {"course_name": "Data Structures", "course_code":
"CS101", "credits": 4}
    target = {"course_name": "Art History", "course_code": "AH101",
"credits": 3}

    result = compute_course_match(
        source_course=source,
        target_course=target,
        source_ku_ids={1},
        target_ku_ids={99},
    )
    assert result.decision == "Not Equivalent"

```

```

=====
=====
FILE: tests/test_pipeline.py
=====
=====

```

```

import pandas as pd

from engine.data_loader import DataBundle
from engine.pipeline import match_transcript_to_institution

def test_pipeline_returns_ranked_matches():
    courses = pd.DataFrame(
        [
            {"course_id": 1, "institution_id": 1, "program_id": 1,
"course_name": "Operating Systems", "course_code": "CSIA 317",
"credits": 3},
            {"course_id": 2, "institution_id": 4, "program_id": 3,
"course_name": "Operating Systems", "course_code": "IT317", "credits":
3},
            {"course_id": 3, "institution_id": 4, "program_id": 3,
"course_name": "Web Development", "course_code": "SD102", "credits":
3},
        ]
    )
    bundle = DataBundle(

```

```

        courses=courses,
        course_ku=pd.DataFrame([{"course_id": 1, "ku_id": 12},
{"course_id": 2, "ku_id": 12}, {"course_id": 3, "ku_id": 4}]),
        knowledge_units=pd.DataFrame([{"ku_id": 12}]),
        institutions=pd.DataFrame([{"institution_id": 1},
{"institution_id": 4}]),
        programs=pd.DataFrame([{"program_id": 1}, {"program_id": 3}]),
        transfer_requests=pd.DataFrame(),
        students=pd.DataFrame(),
        course_syllabus=pd.DataFrame(
            [
                {"course_id": 1, "syllabus_text": "memory scheduling
processes"},
                {"course_id": 2, "syllabus_text": "processes
scheduling memory"},
                {"course_id": 3, "syllabus_text": "html css
javascript"},
            ]
        ),
    )
    transcript_data = {
        "classes": [{"course": "CSIA 317", "title": "Operating
Systems", "grade": "A", "credits": 3, "term": "Fall 2025"}]
    }

    result = match_transcript_to_institution(
        transcript_data=transcript_data,
        target_institution_id=4,
        data_bundle=bundle,
        top_k=2,
    )

    assert len(result) == 2
    top = result.iloc[0]
    assert top["target_course_code"] == "IT317"

```